MICROSAR Classic VStdLib

Technical Reference

Generic implementation of the Vector Standard Library

Version 2.05.00

Author	vismte
Status	Released

Document Information

History

Author	Date	Version	Remarks
vistkr	2015-05-04	1.00.00	Creation
vistkr	2016-04-12	1.00.01	Update to new CI, no changes in content
visgaz	2021-01-18	2.00.00	Added bit search feature (CAN-1574) Made formal adaptions
visgaz	2021-03-15	2.01.00	Added memory comparison (CAN-1579) and (de-) serialization feature (CAN-1577)
visgaz	2021-04-12	2.02.00	Refactored description of memory comparison (CAN-2834) and (de-)serialization feature (CAN-2831) Added endianness conversion feature (CAN-1576)
visgaz visnaj	2022-04-22	2.03.00	Added memory copy with endianness conversion feature (CANCORE-455) Renamed MICROSAR to MICROSAR Classic Refactored description for endianness conversion and (de-)serialization feature Added memory check feature (CANHAL -1215)
visgaz	2023-01-25	2.04.00	Extended API description for memory initialization, memory copy, memory comparison, serialization and deserialization feature (CANCORE-1379)
vismte	2023-03-21	2.05.00	Added bit copy feature (CANCORE-1868)

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	List of Basic Software Modules	R4.2.1
[2]	AUTOSAR	Specification of Default Error Tracer	R4.2.1



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Introduction 11
1.1 Architecture Overview 11

2 Functional Description 12
2.1 Features 12
2.2 Initialization 12
2.3 Main Function 12
2.4 Memory copy with endianness conversion services 12
2.5 Bit copy services 13
2.6 Error Handling 13
2.6.1 Development Error Reporting 13
2.6.2 Production Code Error Reporting 15

3 Integration 16
3.1 Embedded Implementation 16
3.2 Include Structure 16
3.3 Critical Sections 17
3.4 Compiler Abstraction and Memory Mapping 17
3.5 Integration Notes 18

4 API Description 19
4.1 Type Definitions 19
4.2 Services provided by VStdLib 20
4.2.1 Version information 20
4.2.1.1 VStdLib_GetVersionInfo 20
4.2.2 Memory initialization 21
4.2.2.1 VStdLib_MemClr 21
4.2.2.2 VStdLib_MemClrMacro 22
4.2.2.3 VStdLib_MemSet 23
4.2.2.4 VStdLib_MemSetMacro 24
4.2.3 Memory copy 25
4.2.3.1 VStdLib_MemCpy 25
4.2.3.2 VStdLib_MemCpy16 26
4.2.3.3 VStdLib_MemCpy32 27
4.2.3.4 VStdLib_MemCpy_s 28
4.2.3.5 VStdLib_MemCpyMacro 29
4.2.3.6 VStdLib_MemCpyMacro_s 30
4.2.4 Memory copy with endianness conversion 31
4.2.4.1 VStdLib_MemCpySwapEndian 31

4.2.4.2 VStdLib_MemCpySwapEndian_s 32
4.2.4.3 VStdLib_MemCpyEndianPlatformToBig 33
4.2.4.4 VStdLib_MemCpyEndianPlatformToBig_s 34
4.2.4.5 VStdLib_MemCpyEndianPlatformToLittle 35
4.2.4.6 VStdLib_MemCpyEndianPlatformToLittle_s 36
4.2.4.7 VStdLib_MemCpyEndianBigToPlatform 37
4.2.4.8 VStdLib_MemCpyEndianBigToPlatform_s 38
4.2.4.9 VStdLib_MemCpyEndianLittleToPlatform 39
4.2.4.10 VStdLib_MemCpyEndianLittleToPlatform_s 40
4.2.5 Bit copy 41
4.2.5.1 VStdLib_BitCpySawBigToBigEndian 41
4.2.5.2 VStdLib_BitCpySawBigToLittleEndian 42
4.2.5.3 VStdLib_BitCpySawLittleToBigEndian 43
4.2.5.4 VStdLib_BitCpySawLittleToLittleEndian 44
4.2.5.5 VStdLib_BitCpySawBigToBigEndian_s 45
4.2.5.6 VStdLib_BitCpySawBigToLittleEndian_s 46
4.2.5.7 VStdLib_BitCpySawLittleToBigEndian_s 47
4.2.5.8 VStdLib_BitCpySawLittleToLittleEndian_s 48
4.2.5.9 VStdLib_BitCpyMonBigToBigEndian 49
4.2.5.10 VStdLib_BitCpyMonBigToLittleEndian 50
4.2.5.11 VStdLib_BitCpyMonLittleToBigEndian 51
4.2.5.12 VStdLib_BitCpyMonLittleToLittleEndian 52
4.2.5.13 VStdLib_BitCpyMonBigToBigEndian_s 53
4.2.5.14 VStdLib_BitCpyMonBigToLittleEndian_s 54
4.2.5.15 VStdLib_BitCpyMonLittleToBigEndian_s 55
4.2.5.16 VStdLib_BitCpyMonLittleToLittleEndian_s 56
4.2.6 Memory comparison 57
4.2.6.1 VStdLib_MemCmp 57
4.2.6.2 VStdLib_MemCheck 58
4.2.7 Bit search 59
4.2.7.1 VStdLib_GetHighestBitPosOne8 59
4.2.7.2 VStdLib_GetHighestBitPosOne16 59
4.2.7.3 VStdLib_GetHighestBitPosOne32 60
4.2.7.4 VStdLib_GetHighestBitPosOne64 60
4.2.7.5 VStdLib_GetHighestBitPosZero8 61
4.2.7.6 VStdLib_GetHighestBitPosZero16 61
4.2.7.7 VStdLib_GetHighestBitPosZero32 62
4.2.7.8 VStdLib_GetHighestBitPosZero64 62
4.2.7.9 VStdLib_GetLowestBitPosOne8 63
4.2.7.10 VStdLib_GetLowestBitPosOne16 63
4.2.7.11 VStdLib_GetLowestBitPosOne32 64

4.2.7.12 VStdLib_GetLowestBitPosOne64 64
4.2.7.13 VStdLib_GetLowestBitPosZero8 65
4.2.7.14 VStdLib_GetLowestBitPosZero16 65
4.2.7.15 VStdLib_GetLowestBitPosZero32 66
4.2.7.16 VStdLib_GetLowestBitPosZero64 66
4.2.7.17 VStdLib_GetCountLeadingOnes8 67
4.2.7.18 VStdLib_GetCountLeadingOnes16 67
4.2.7.19 VStdLib_GetCountLeadingOnes32 68
4.2.7.20 VStdLib_GetCountLeadingOnes64 68
4.2.7.21 VStdLib_GetCountLeadingZeros8 69
4.2.7.22 VStdLib_GetCountLeadingZeros16 69
4.2.7.23 VStdLib_GetCountLeadingZeros32 70
4.2.7.24 VStdLib_GetCountLeadingZeros64 70
4.2.7.25 VStdLib_GetCountTrailingOnes8 71
4.2.7.26 VStdLib_GetCountTrailingOnes16 71
4.2.7.27 VStdLib_GetCountTrailingOnes32 72
4.2.7.28 VStdLib_GetCountTrailingOnes64 72
4.2.7.29 VStdLib_GetCountTrailingZeros8 73
4.2.7.30 VStdLib_GetCountTrailingZeros16 73
4.2.7.31 VStdLib_GetCountTrailingZeros32 74
4.2.7.32 VStdLib_GetCountTrailingZeros64 74
4.2.8 Serialization 75
4.2.8.1 VStdLib_ConvertUint16ToUint8ArrayBigEndian 75
4.2.8.2 VStdLib_ConvertSint16ToUint8ArrayBigEndian 76
4.2.8.3 VStdLib_ConvertUint16ToUint8ArrayLittleEndian 77
4.2.8.4 VStdLib_ConvertSint16ToUint8ArrayLittleEndian 78
4.2.8.5 VStdLib_ConvertUint32ToUint8ArrayBigEndian 79
4.2.8.6 VStdLib_ConvertSint32ToUint8ArrayBigEndian 80
4.2.8.7 VStdLib_ConvertUint32ToUint8ArrayLittleEndian 81
4.2.8.8 VStdLib_ConvertSint32ToUint8ArrayLittleEndian 82
4.2.8.9 VStdLib_ConvertUint64ToUint8ArrayBigEndian 83
4.2.8.10 VStdLib_ConvertSint64ToUint8ArrayBigEndian 84
4.2.8.11 VStdLib_ConvertUint64ToUint8ArrayLittleEndian 85
4.2.8.12 VStdLib_ConvertSint64ToUint8ArrayLittleEndian 86
4.2.9 Deserialization 87
4.2.9.1 VStdLib_ConvertUint8ArrayToUint16BigEndian 87
4.2.9.2 VStdLib_ConvertUint8ArrayToSint16BigEndian 88
4.2.9.3 VStdLib_ConvertUint8ArrayToUint16LittleEndian 89
4.2.9.4 VStdLib_ConvertUint8ArrayToSint16LittleEndian 90
4.2.9.5 VStdLib_ConvertUint8ArrayToUint32BigEndian 91
4.2.9.6 VStdLib_ConvertUint8ArrayToSint32BigEndian 92

4.2.9.7 VStdLib_ConvertUint8ArrayToUint32LittleEndian 93
4.2.9.8 VStdLib_ConvertUint8ArrayToSint32LittleEndian 94
4.2.9.9 VStdLib_ConvertUint8ArrayToUint64BigEndian 95
4.2.9.10 VStdLib_ConvertUint8ArrayToSint64BigEndian 96
4.2.9.11 VStdLib_ConvertUint8ArrayToUint64LittleEndian 97
4.2.9.12 VStdLib_ConvertUint8ArrayToSint64LittleEndian 98
4.2.10 Endianness swap 99
4.2.10.1 VStdLib_SwapEndianUint16 99
4.2.10.2 VStdLib_SwapEndianSint16 99
4.2.10.3 VStdLib_SwapEndianUint32 100
4.2.10.4 VStdLib_SwapEndianSint32 100
4.2.10.5 VStdLib_SwapEndianUint64 101
4.2.10.6 VStdLib_SwapEndianSint64 101
4.2.11 Endianness conversion 102
4.2.11.1 VStdLib_ConvertEndianPlatformToBigUint16 102
4.2.11.2 VStdLib_ConvertEndianPlatformToBigSint16 102
4.2.11.3 VStdLib_ConvertEndianPlatformToLittleUint16 103
4.2.11.4 VStdLib_ConvertEndianPlatformToLittleSint16 103
4.2.11.5 VStdLib_ConvertEndianBigToPlatformUint16 104
4.2.11.6 VStdLib_ConvertEndianBigToPlatformSint16 104
4.2.11.7 VStdLib_ConvertEndianLittleToPlatformUint16 105
4.2.11.8 VStdLib_ConvertEndianLittleToPlatformSint16 105
4.2.11.9 VStdLib_ConvertEndianPlatformToBigUint32 106
4.2.11.10 VStdLib_ConvertEndianPlatformToBigSint32 106
4.2.11.11 VStdLib_ConvertEndianPlatformToLittleUint32 107
4.2.11.12 VStdLib_ConvertEndianPlatformToLittleSint32 107
4.2.11.13 VStdLib_ConvertEndianBigToPlatformUint32 108
4.2.11.14 VStdLib_ConvertEndianBigToPlatformSint32 108
4.2.11.15 VStdLib_ConvertEndianLittleToPlatformUint32 109
4.2.11.16 VStdLib_ConvertEndianLittleToPlatformSint32 109
4.2.11.17 VStdLib_ConvertEndianPlatformToBigUint64 110
4.2.11.18 VStdLib_ConvertEndianPlatformToBigSint64 110
4.2.11.19 VStdLib_ConvertEndianPlatformToLittleUint64 111
4.2.11.20 VStdLib_ConvertEndianPlatformToLittleSint64 111
4.2.11.21 VStdLib_ConvertEndianBigToPlatformUint64 112
4.2.11.22 VStdLib_ConvertEndianBigToPlatformSint64 112
4.2.11.23 VStdLib_ConvertEndianLittleToPlatformUint64 113
4.2.11.24 VStdLib_ConvertEndianLittleToPlatformSint64 113
4.3 Services used by VStdLib 114

5 Configuration 115

5.1 Configuration Variants 115
5.2 Configuration manually in Header Files 115
5.2.1 General configuration 115
5.2.2 Additional configuration when using library functions 119
5.2.2.1 Memory services 119
5.2.2.2 Bit search services 120
5.2.2.3 Deserialization and serialization services 120
5.2.2.4 Endianness conversion services 120

6 Abbreviations 121

7 Contact 122

Illustrations

Figure 1-1 Interfaces to adjacent modules of the VStdLib 11
Figure 3-1 Include Structure 16

Tables

Table 2-1 Supported Features 12
Table 2-2 Bit numbering 13
Table 2-3 Service IDs 14
Table 2-4 Errors reported to DET 15
Table 3-1 Implementation files 16
Table 3-2 Compiler Abstraction and Memory Mapping 17
Table 4-1 VStdLib_GetVersionInfo 20
Table 4-2 VStdLib_MemClr 21
Table 4-3 VStdLib_MemClrMacro 22
Table 4-4 VStdLib_MemSet 23
Table 4-5 VStdLib_MemSetMacro 24
Table 4-6 VStdLib_MemCpy 25
Table 4-7 VStdLib_MemCpy16 26
Table 4-8 VStdLib_MemCpy32 27
Table 4-9 VStdLib_MemCpy_s 28
Table 4-10 VStdLib_MemCpyMacro 29
Table 4-11 VStdLib_MemCpyMacro_s 30
Table 4-12 VStdLib_MemCpySwapEndian 31
Table 4-13 VStdLib_MemCpySwapEndian_s 33
Table 4-14 VStdLib_MemCpyEndianPlatformToBig 33
Table 4-15 VStdLib_MemCpyEndianPlatformToBig_s 35
Table 4-16 VStdLib_MemCpyEndianPlatformToLittle 35
Table 4-17 VStdLib_MemCpyEndianPlatformToLittle_s 37
Table 4-18 VStdLib_MemCpyEndianBigToPlatform 37
Table 4-19 VStdLib_MemCpyEndianBigToPlatform_s 39
Table 4-20 VStdLib_MemCpyEndianLittleToPlatform 39
Table 4-21 VStdLib_MemCpyEndianLittleToPlatform_s 41
Table 4-22 VStdLib_BitCpySawBigToBigEndian 41
Table 4-23 VStdLib_BitCpySawBigToLittleEndian 42
Table 4-24 VStdLib_BitCpySawLittleToBigEndian 43
Table 4-25 VStdLib_BitCpySawLittleToLittleEndian 44
Table 4-26 VStdLib_BitCpySawBigToBigEndian_s 45
Table 4-27 VStdLib_BitCpySawBigToLittleEndian_s 46
Table 4-28 VStdLib_BitCpySawLittleToBigEndian_s 47
Table 4-29 VStdLib_BitCpySawLittleToLittleEndian_s 48
Table 4-30 VStdLib_BitCpyMonBigToBigEndian 49
Table 4-31 VStdLib_BitCpyMonBigToLittleEndian 50
Table 4-32 VStdLib_BitCpyMonLittleToBigEndian 51
Table 4-33 VStdLib_BitCpyMonLittleToLittleEndian 52
Table 4-34 VStdLib_BitCpyMonBigToBigEndian_s 53
Table 4-35 VStdLib_BitCpyMonBigToLittleEndian_s 54
Table 4-36 VStdLib_BitCpyMonLittleToBigEndian_s 55
Table 4-37 VStdLib_BitCpyMonLittleToLittleEndian_s 56
Table 4-38 VStdLib_MemCmp 57
Table 4-39 VStdLib_MemCheck 58
Table 4-40 VStdLib_GetHighestBitPosOne8 59

Table 4-41 VStdLib_GetHighestBitPosOne16 59
Table 4-42 VStdLib_GetHighestBitPosOne32 60
Table 4-43 VStdLib_GetHighestBitPosOne64 60
Table 4-44 VStdLib_GetHighestBitPosZero8 61
Table 4-45 VStdLib_GetHighestBitPosZero16 61
Table 4-46 VStdLib_GetHighestBitPosZero32 62
Table 4-47 VStdLib_GetHighestBitPosZero8 62
Table 4-48 VStdLib_GetLowestBitPosOne8 63
Table 4-49 VStdLib_GetLowestBitPosOne16 63
Table 4-50 VStdLib_GetLowestBitPosOne32 64
Table 4-51 VStdLib_GetLowestBitPosOne64 64
Table 4-52 VStdLib_GetLowestBitPosZero8 65
Table 4-53 VStdLib_GetLowestBitPosZero16 65
Table 4-54 VStdLib_GetLowestBitPosZero32 66
Table 4-55 VStdLib_GetLowestBitPosZero64 66
Table 4-56 VStdLib_GetCountLeadingOnes8 67
Table 4-57 VStdLib_GetCountLeadingOnes16 67
Table 4-58 VStdLib_GetCountLeadingOnes32 68
Table 4-59 VStdLib_GetCountLeadingOnes64 68
Table 4-60 VStdLib_GetCountLeadingZeros8 69
Table 4-61 VStdLib_GetCountLeadingZeros16 69
Table 4-62 VStdLib_GetCountLeadingZeros32 70
Table 4-63 VStdLib_GetCountLeadingZeros64 70
Table 4-64 VStdLib_GetCountTrailingOnes8 71
Table 4-65 VStdLib_GetCountTrailingOnes16 71
Table 4-66 VStdLib_GetCountTrailingOnes32 72
Table 4-67 VStdLib_GetCountTrailingOnes64 72
Table 4-68 VStdLib_GetCountTrailingZeros8 73
Table 4-69 VStdLib_GetCountTrailingZeros16 73
Table 4-70 VStdLib_GetCountTrailingZeros32 74
Table 4-71 VStdLib_GetCountTrailingZeros64 74
Table 4-72 VStdLib_ConvertUint16ToUint8ArrayBigEndian 75
Table 4-73 VStdLib_ConvertSint16ToUint8ArrayBigEndian 76
Table 4-74 VStdLib_ConvertUint16ToUint8ArrayLittleEndian 77
Table 4-75 VStdLib_ConvertSint16ToUint8ArrayLittleEndian 78
Table 4-76 VStdLib_ConvertUint32ToUint8ArrayBigEndian 79
Table 4-77 VStdLib_ConvertSint32ToUint8ArrayBigEndian 80
Table 4-78 VStdLib_ConvertUint32ToUint8ArrayLittleEndian 81
Table 4-79 VStdLib_ConvertSint32ToUint8ArrayLittleEndian 82
Table 4-80 VStdLib_ConvertUint64ToUint8ArrayBigEndian 83
Table 4-81 VStdLib_ConvertSint64ToUint8ArrayBigEndian 84
Table 4-82 VStdLib_ConvertUint64ToUint8ArrayLittleEndian 85
Table 4-83 VStdLib_ConvertSint64ToUint8ArrayLittleEndian 86
Table 4-84 VStdLib_ConvertUint8ArrayToUint16BigEndian 87
Table 4-85 VStdLib_ConvertUint8ArrayToSint16BigEndian 88
Table 4-86 VStdLib_ConvertUint8ArrayToUint16LittleEndian 89
Table 4-87 VStdLib_ConvertUint8ArrayToSint16LittleEndian 90
Table 4-88 VStdLib_ConvertUint8ArrayToUint32BigEndian 91
Table 4-89 VStdLib_ConvertUint8ArrayToSint32BigEndian 92
Table 4-90 VStdLib_ConvertUint8ArrayToUint32LittleEndian 93
Table 4-91 VStdLib_ConvertUint8ArrayToSint32LittleEndian 94
Table 4-92 VStdLib_ConvertUint8ArrayToUint64BigEndian 95
Table 4-93 VStdLib_ConvertUint8ArrayToSint64BigEndian 96
Table 4-94 VStdLib_ConvertUint8ArrayToUint64LittleEndian 97

Table 4-95 VStdLib_ConvertUint8ArrayToSint64LittleEndian 98
Table 4-96 VStdLib_SwapEndianUint16 99
Table 4-97 VStdLib_SwapEndianSint16 99
Table 4-98 VStdLib_SwapEndianUint32 100
Table 4-99 VStdLib_SwapEndianSint32 100
Table 4-100 VStdLib_SwapEndianUint64 101
Table 4-101 VStdLib_SwapEndianSint64 101
Table 4-102 VStdLib_ConvertEndianPlatformToBigUint16 102
Table 4-103 VStdLib_ConvertEndianPlatformToBigSint16 102
Table 4-104 VStdLib_ConvertEndianPlatformToLittleUint16 103
Table 4-105 VStdLib_ConvertEndianPlatformToLittleSint16 103
Table 4-106 VStdLib_ConvertEndianBigToPlatformUint16 104
Table 4-107 VStdLib_ConvertEndianBigToPlatformSint16 104
Table 4-108 VStdLib_ConvertEndianLittleToPlatformUint16 105
Table 4-109 VStdLib_ConvertEndianLittleToPlatformSint16 105
Table 4-110 VStdLib_ConvertEndianPlatformToBigUint32 106
Table 4-111 VStdLib_ConvertEndianPlatformToBigSint32 106
Table 4-112 VStdLib_ConvertEndianPlatformToLittleUint32 107
Table 4-113 VStdLib_ConvertEndianPlatformToLittleSint32 107
Table 4-114 VStdLib_ConvertEndianBigToPlatformUint32 108
Table 4-115 VStdLib_ConvertEndianBigToPlatformSint32 108
Table 4-116 VStdLib_ConvertEndianLittleToPlatformUint32 109
Table 4-117 VStdLib_ConvertEndianLittleToPlatformSint32 109
Table 4-118 VStdLib_ConvertEndianPlatformToBigUint64 110
Table 4-119 VStdLib_ConvertEndianPlatformToBigSint64 110
Table 4-120 VStdLib_ConvertEndianPlatformToLittleUint64 111
Table 4-121 VStdLib_ConvertEndianPlatformToLittleSint64 111
Table 4-122 VStdLib_ConvertEndianBigToPlatformUint64 112
Table 4-123 VStdLib_ConvertEndianBigToPlatformSint64 112
Table 4-124 VStdLib_ConvertEndianLittleToPlatformUint64 113
Table 4-125 VStdLib_ConvertEndianLittleToPlatformSint64 113
Table 4-126 Services used by VStdLib 114
Table 5-1 General configuration 118
Table 6-1 Abbreviations 121

1 Introduction

This document describes the functionality, API and configuration of the generic Vector Standard Library (VStdLib).

Supported Configuration Variants:	pre-compile	
Vendor ID:	VSTDLIB_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VSTDLIB_MODULE_ID	255 decimal (according to [1])

The VStdLib provides a set of services with a generic hardware independent implementation used by several MICROSAR Classic BSW components. The services can also be mapped to project specific implementations for optimization reasons.

1.1 Architecture Overview

The next figure shows the interfaces to adjacent modules of the VStdLib. These interfaces are described in chapter 4.

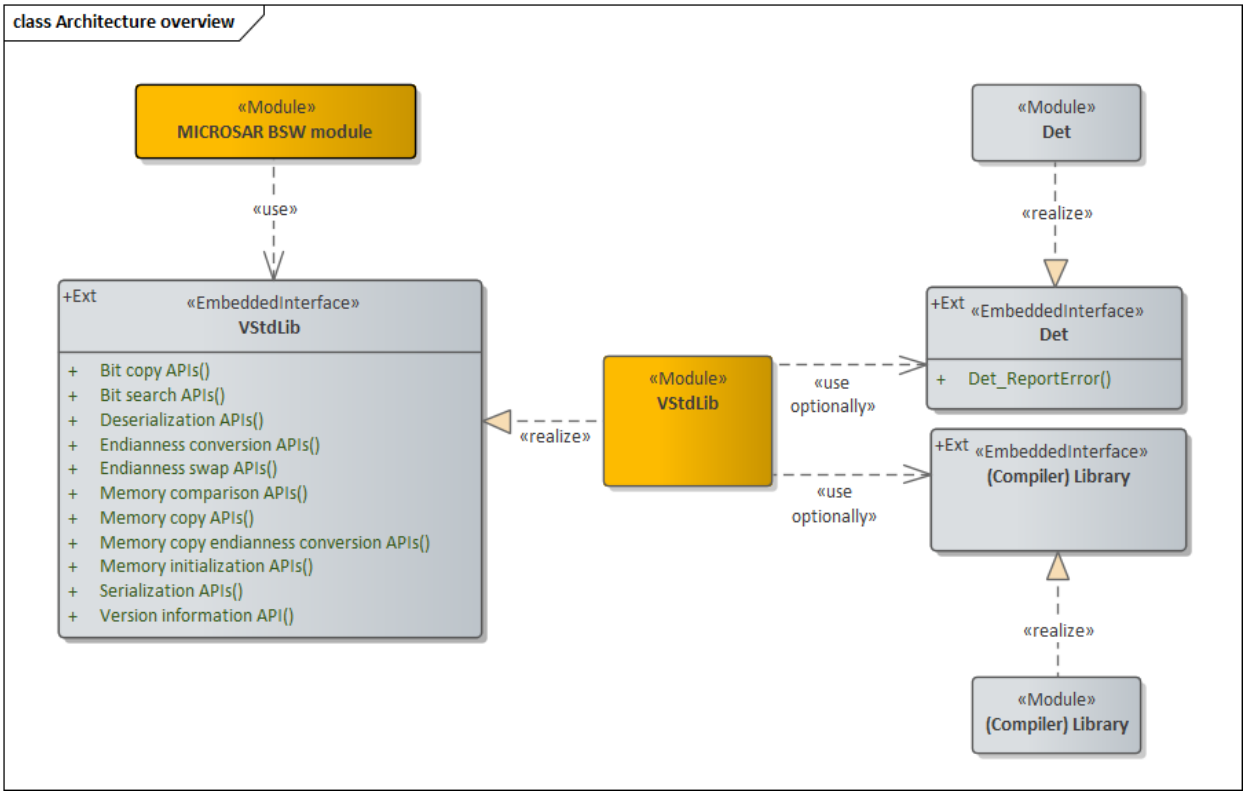


Figure 1-1 Interfaces to adjacent modules of the VStdLib

2 Functional Description

2.1 Features

The features listed in the following table cover the functionality specified for the VStdLib.

Supported Features
Version information
Memory initialization services
Memory copy services
Memory copy with endianness conversion services
Bit copy services
Memory comparison services
Bit search services
Serialization services
Deserialization services
Endianness swap services
Endianness conversion services

Table 2-1 Supported Features

2.2 Initialization

The VStdLib can be used without any initialization. Therefore, the VStdLib has no initialization function.

2.3 Main Function

The VStdLib provides no main function.

2.4 Memory copy with endianness conversion services

The generic implementation provided by the VStdLib for these services (`VSTDLIB_USE_MEM_CPY_ENDIAN_CONVERSION_LIBRARY_FUNCTION == STD_OFF`) use the following different memory access sizes to destination and source depending on the VStdLib configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt`.

`intType` is 8-bit: Uses `VStdLib_MemCpy()`.

`intType` is 16-bit, 32-bit or 64-bit:

- `VSTDLIB_RUNTIME_OPTIMIZATION == STD_OFF`:

  `intType` (16-bit, 32-bit or 64-bit) dependent access to destination and 8-bit access to source.

- VSTDLIB_RUNTIME_OPTIMIZATION == STD_ON:
  - nCnt < 33 or destination and/or source are not 32-bit aligned: intType (16-bit, 32-bit or 64-bit) dependent access to destination and 8-bit access to source.
  - nCnt >= 33 and destination and source are 32-bit aligned: 32-bit access to destination and source. For remaining bytes intType (16-bit, 32-bit or 64-bit) dependent access to destination and 8-bit access to source.

2.5 Bit copy services

The services provided by the VStdLib support two different bit numbering methods: sawtooth and monotone. The sawtooth bit numbering is in line with AUTOSAR and is decreasing within a byte. On the other hand, monotone bit numbering is increasing within a byte. The next table shows based on two bytes how the bit numbering has to look like according to the bit numbering method.

Bit numbering	Byte 0	Byte 1
Sawtooth	7 6 5 4 3 2 1 0	15 14 13 12 11 10 9 8
Monotone	0 1 2 3 4 5 6 7	8 9 10 11 12 13 14 15

Table 2-2 Bit numbering

2.6 Error Handling

2.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service Det_ReportError() as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter VSTDLIB_DEV_ERROR_REPORT == STD_ON).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service Det_ReportError().

The reported VStdLib ID is 255.

The service IDs identify the services which are described in section 4.2. The following table presents the service IDs and the related services:

Service ID	Service
VSTDLIB_SID_MEM_SET (0x00)	VStdLib_MemClr(), VStdLib_MemSet()
VSTDLIB_SID_MEM_COPY (0x01)	VStdLib_MemCpy(), VStdLib_MemCpyEndianPlatformToBig(), VStdLib_MemCpyEndianPlatformToLittle(), VStdLib_MemCpyEndianBigToPlatform(), VStdLib_MemCpyEndianLittleToPlatform()
VSTDLIB_SID_MEM_COPY_16 (0x02)	VStdLib_MemCpy16()
VSTDLIB_SID_MEM_COPY_32 (0x03)	VStdLib_MemCpy32()

Service ID	Service
VSTDLIB_SID_MEM_COPY_S (0x04)	VStdLib_MemCpy_s(), VStdLib_MemCpyEndianPlatformToBig_s(), VStdLib_MemCpyEndianPlatformToLittle_s(), VStdLib_MemCpyEndianBig ToPlatform_s(), VStdLib_MemCpyEndianLittleToPlatform_s()
VSTDLIB_SID_GET_VERSION_INFO (0x05)	VStdLib_GetVersionInfo()
VSTDLIB_SID_MEM_CMP (0x06)	VStdLib_MemCmp()
VSTDLIB_SID_CNV_UINT16_TO_ARRAY_BE (0x07)	VStdLib_ConvertUint16ToUint8ArrayBigEndian(), VStdLib_ConvertSint16ToUint8ArrayBigEndian()
VSTDLIB_SID_CNV_UINT16_TO_ARRAY_LE (0x08)	VStdLib_ConvertUint16ToUint8ArrayLittleEndian(), VStdLib_ConvertSint16ToSint8ArrayLittleEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT16_BE (0x09)	VStdLib_ConvertUint8ArrayToUint16BigEndian(), VStdLib_ConvertUint8ArrayToSint16BigEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT16_LE (0x0A)	VStdLib_ConvertUint8ArrayToUint16LittleEndian(), VStdLib_ConvertUint8ArrayToSint16LittleEndian()
VSTDLIB_SID_CNV_UINT32_TO_ARRAY_BE (0x0B)	VStdLib_ConvertUint32ToUint8ArrayBigEndian(), VStdLib_ConvertSint32ToUint8ArrayBigEndian()
VSTDLIB_SID_CNV_UINT32_TO_ARRAY_LE (0x0C)	VStdLib_ConvertUint32ToUint8ArrayLittleEndian(), VStdLib_ConvertSint32ToUint8ArrayLittleEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT32_BE (0x0D)	VStdLib_ConvertUint8ArrayToUint32BigEndian(), VStdLib_ConvertUint8ArrayToSint32BigEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT32_LE (0x0E)	VStdLib_ConvertUint8ArrayToUint32LittleEndian(), VStdLib_ConvertUint8ArrayToSint32LittleEndian()
VSTDLIB_SID_CNV_UINT64_TO_ARRAY_BE (0x0F)	VStdLib_ConvertUint64ToUint8ArrayBigEndian(), VStdLib_ConvertSint64ToUint8ArrayBigEndian()
VSTDLIB_SID_CNV_UINT64_TO_ARRAY_LE (0x10)	VStdLib_ConvertUint64ToUint8ArrayLittleEndian(), VStdLib_ConvertSint64ToUint8ArrayLittleEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT64_BE (0x11)	VStdLib_ConvertUint8ArrayToUint64BigEndian(), VStdLib_ConvertUint8ArrayToSint64BigEndian()
VSTDLIB_SID_CNV_ARRAY_TO_UINT64_LE (0x12)	VStdLib_ConvertUint8ArrayToUint64LittleEndian(), VStdLib_ConvertUint8ArrayToSint64LittleEndian()
VSTDLIB_SID_MEM_COPY_SWAP_ENDIAN (0x13)	VStdLib_MemCpySwapEndian(), VStdLib_MemCpyEndianPlatformToBig(), VStdLib_MemCpyEndianPlatformToLittle(), VStdLib_MemCpyEndianBigToPlatform(), VStdLib_MemCpyEndianLittleToPlatform()
VSTDLIB_SID_MEM_COPY_SWAP_ENDIAN_S(0x14)	VStdLib_MemCpySwapEndian_s(), VStdLib_MemCpyEndianPlatformToBig_s(), VStdLib_MemCpyEndianPlatformToLittle_s(), VStdLib_MemCpyEndianBig ToPlatform_s(), VStdLib_MemCpyEndianLittleToPlatform_s()
VSTDLIB_SID_MEM_CHECK (0x15)	VStdLib_MemCheck()
VSTDLIB_SID_BIT_COPY_SAW (0x16)	VStdLib_BitCpySawBigToBigEndian(), VStdLib_BitCpySawBigToLittleEndian(), VStdLib_BitCpySawLittleToBigEndian(), VStdLib_BitCpySawLittleToLittleEndian()
VSTDLIB_SID_BIT_COPY_MON (0x17)	VStdLib_BitCpyMonBigToBigEndian(), VStdLib_BitCpyMonBigToLittleEndian(), VStdLib_BitCpyMonLittleToBigEndian(), VStdLib_BitCpyMonLittleToLittleEndian()
VSTDLIB_SID_BIT_COPY_SAW_S (0x18)	VStdLib_BitCpySawBigToBigEndian_s(), VStdLib_BitCpySawBigToLittleEndian_s(), VStdLib_BitCpySawLittleToBigEndian_s(), VStdLib_BitCpySawLittleToLittleEndian_s()
VSTDLIB_SID_BIT_COPY_MON_S (0x19)	VStdLib_BitCpyMonBigToBigEndian_s(), VStdLib_BitCpyMonBigToLittleEndian_s(), VStdLib_BitCpyMonLittleToBigEndian_s(), VStdLib_BitCpyMonLittleToLittleEndian_s()

Table 2-3 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
VSTDLIB_E_PARAM_POINTER (0x01)	API used with invalid pointer parameter (e.g. NULL).
VSTDLIB_E_PARAM_SIZE (0x02)	API used with invalid destination buffer size.
VSTDLIB_E_PARAM_INTTYPE (0x03)	API used with invalid integer type parameter.

Table 2-4 Errors reported to DET

2.6.2 Production Code Error Reporting

No production code error reporting is supported.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR Classic VStdLib into an application environment of an ECU.

3.1 Embedded Implementation

The delivery of the VStdLib contains these source code files:

File Name	Description	Integration Tasks
vstdlib.c	This is the source file of the VStdLib. It contains the hardware independent implementation of the public APIs.	-
vstdlib.h	This is the header file of the VStdLib. It contains the public (inline) APIs and global data of the VStdLib. It has to be included by other modules to access the APIs of the VStdLib.	-
_VStdLib_Cfg.h	Template file that can be adapted by the user. This file contains the project specific configuration of the VStdLib. It also provides the possibility of overwriting the public APIs with project specific optimized implementations.	See chapter 5 for details.

Table 3-1 Implementation files

3.2 Include Structure

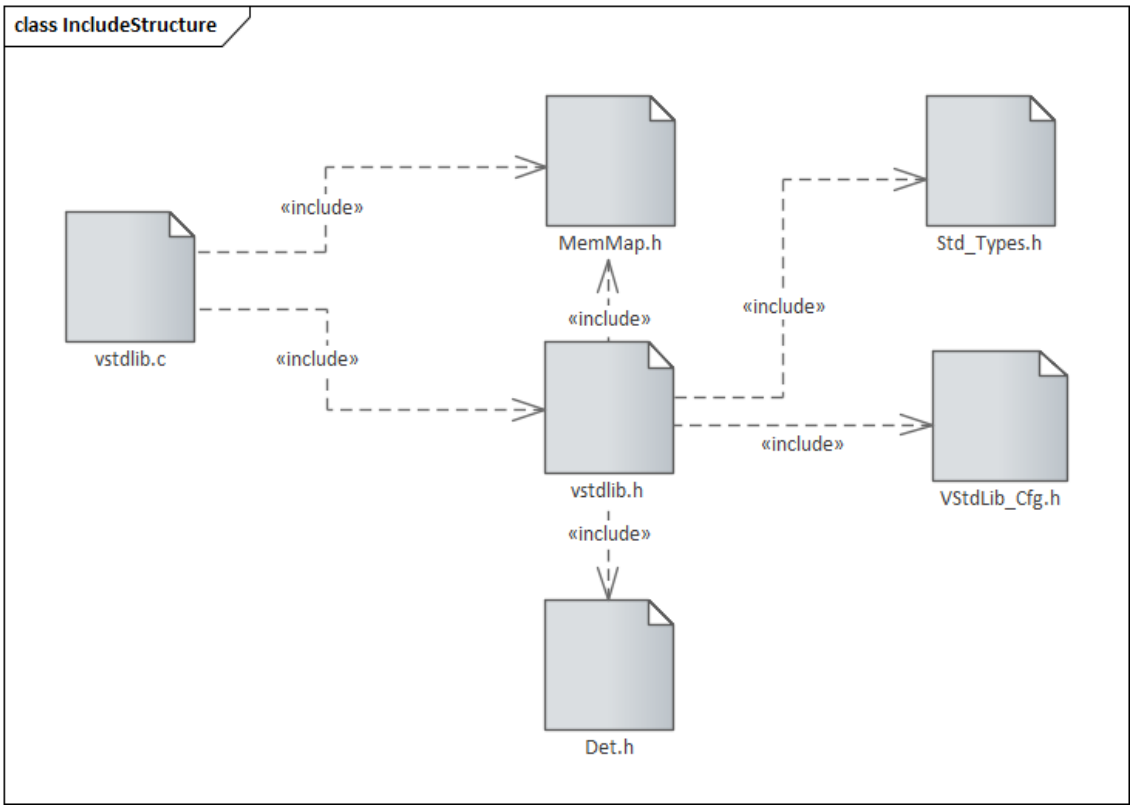


Figure 3-1 Include Structure

3.3 Critical Sections

No critical sections are implemented.

3.4 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions of the VStdLib and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	VSTDLIB_CODE	VSTDLIB_APPL_VAR	VSTDLIB_VAR_FAR	VSTDLIB_CONST
VSTDLIB_START_SEC_CODE VSTDLIB_STOP_SEC_CODE	■			
VSTDLIB_START_SEC_CONST_8BIT VSTDLIB_STOP_SEC_CONST_8BIT				■

Table 3-2 Compiler Abstraction and Memory Mapping

Due to the AUTOSAR compiler and memory abstraction the location of individual data and the width of pointers are not known during development time. Hence the compiler abstraction definition VSTDLIB_APPL_VAR refers to variables which are defined by the application and not handled by the VStdLib memory mapping.

The function implementation of all services as described in section 4.2 abstracts all pointer arguments (independently of the target type) with VSTDLIB_VAR_FAR to allow that all data can be handled by far accesses. The macro implementation performs an in-place access, thus the unaltered pointer class of the caller is used.



Note
For most 32-bit MCUs it is not necessary to give any qualifiers with VSTDLIB_VAR_FAR to change the pointer class as these platforms commonly use 32-bit pointers that can address the whole memory by default. Please note that both RAM and ROM are accessed with this qualifier and an external implementation (see section 5.2.1) has to be used if distinct pointers have to be qualified individually on the used target platform.

3.5 Integration Notes

- Avoid overlapping destination and source memory areas when using the copy services as this may lead to unexpected results.
- Consider side effects if macros are used for the memory initialization and memory copy services: If either destination or source pointer is retrieved using a call to a function, this function might be invoked for `nCnt` times. It is recommended to use temporary pointers to avoid any side effects.
- The VStdLib generic hardware independent implementation for the memory initialization, memory copy, memory copy with endianness conversion and memory comparison services is optimized for 32-bit MCUs. When using a controller with a smaller bit width it is highly recommended to use an external implementation that is optimized for this architecture (see `VSTDLIB_USE_LIBRARY_FUNCTIONS` / `VSTDLIB_USE_MEM_CPY_ENDIAN_CONVERSION_LIBRARY_FUNCTION` / `VSTDLIB_USE_MEM_CHECK_LIBRARY_FUNCTION` in section 5.2.1). If large data support is not necessary, define both `VSTDLIB_RUNTIME_OPTIMIZATION` and `VSTDLIB_SUPPORT_LARGE_DATA` to `STD_OFF` as an alternative.
- The VStdLib generic implementation for memory copy with endianness conversion services uses the `VStdLib_MemCpy()`, `VStdLib_SwapEndianUint[16, 32]()` and `VStdLib_ConvertUint8ArrayToUint[16, 32, 64][Big, Little]Endian()` APIs. To optimize the generic implementation these APIs can be mapped to external implementations that are optimized for the target platform (see `VSTDLIB_USE_LIBRARY_FUNCTIONS` / `VSTDLIB_USE_[16, 32]BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS` / `VSTDLIB_USE_[16, 32, 64]BIT_DE_SERIALIATION_LIBRARY_FUNCTIONS` in section 5.2.1).
- The VStdLib generic implementation for bit copy services uses the `VStdLib_MemCpy()` API. To optimize the generic implementation this API can be mapped to an external implementation that is optimized for the target platform (see `VSTDLIB_USE_LIBRARY_FUNCTIONS` in section 5.2.1).

4 API Description

For an interface overview please see Figure 1-1.

4.1 Type Definitions

The `nCnt` parameter of the memory initialization, memory copy, memory copy with endianness conversion, memory comparison and bit copy services is of type `VStdLib_CntType`. The `nDstBitOffsetSaw/nDstBitOffsetMon` parameters and the `nSrcBitOffsetSaw/nSrcBitOffsetMon` parameters of the bit copy services are also of type `VStdLib_CntType`. `VStdLib_CntType` equals `uint32_least`, if `VSTDLIB_SUPPORT_LARGE_DATA` is defined to `STD_ON` (default setting) or `uint16_least` if it is explicitly defined to `STD_OFF`.

The `intType` parameter of type `uint8` of the memory copy with endianness conversion services can only be used with the following values (defines in file `vstdlib.h`):

- `1 (VSTDLIB_INTTYPE_8)` for integer type sint8/uint8
- `2 (VSTDLIB_INTTYPE_16)` for integer type sint16/uint16
- `4 (VSTDLIB_INTTYPE_32)` for integer type sint32/uint32
- `8 (VSTDLIB_INTTYPE_64)` for integer type sint64/uint64 (only if `PLATFORM_SUPPORT_SINT64_UINT64` is defined)

4.2 Services provided by VStdLib

4.2.1 Version information

4.2.1.1 VStdLib_GetVersionInfo

Prototype	
`void` **`VStdLib_GetVersionInfo`** `( Std_VersionInfoType *versioninfo )`	
Parameter	
`versioninfo [out]`	Pointer to where to store the version information, must not be NULL.
Return code	
`void`	none
Functional Description	
Returns the version information of this module. Returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant. > The parameter `versioninfo` has to be valid and reference an object of type `Std_VersionInfoType`. > Configuration Variant(s): `VSTDLIB_VERSION_INFO_API == STD_ON`	
Call context	
> ANY	

Table 4-1 VStdLib_GetVersionInfo

4.2.2 Memory initialization

4.2.2.1 VStdLib_MemClr

Prototype	
`void` **`VStdLib_MemClr`** `( void *pDst, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to be initialized, must not be NULL.
`nCnt [in]`	Number of bytes to initialize, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Initializes memory to zero. Sets `nCnt` bytes starting at `pDst` to zero.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`, else it is realized by a call to `VStdLib_MemSet()` with `nPattern == 0`. The compatible definition `VStdLib_MemClrLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
ANY	

Table 4-2 VStdLib_MemClr

4.2.2.2 VStdLib_MemClrMacro

Prototype	
`void` **`VStdLib_MemClrMacro`** `( AnyPtrType *pDst, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Any typed pointer to the memory location to be initialized, must be aligned corresponding to its type and not be NULL.
`nCnt [in]`	Number of blocks to initialize, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Initializes memory to zero (macro implementation). Sets `nCnt` blocks starting at `pDst` to zero (block-size is given by the type of `pDst`).	
Particularities and Limitations	
> This macro is synchronous. > This macro is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context.	
Call context	
> ANY	

Table 4-3 VStdLib_MemClrMacro

4.2.2.3 VStdLib_MemSet

Prototype	
`void` **`VStdLib_MemSet`** `( void *pDst, uint8 nPattern, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to be initialized, must not be NULL.
`nPattern [in]`	The character to be used to initialize the memory.
`nCnt [in]`	Number of bytes to initialize, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Initializes memory to a specified pattern. Sets `nCnt` bytes starting at `pDst` to the character `nPattern`.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`. The compatible definition `VStdLib_MemSetLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-4 VStdLib_MemSet

4.2.2.4 VStdLib_MemSetMacro

Prototype	
`void` **`VStdLib_MemSetMacro`** `( AnyPtrType *pDst, AnyIntType nPattern, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Any typed pointer to the memory location to be initialized, must be aligned corresponding to its type and not be NULL.
`nPattern [in]`	The pattern to be used to initialize the memory (consider the correlation between its type and the type of `pDst`).
`nCnt [in]`	Number of blocks to initialize, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Initializes memory to a specified pattern (macro implementation). Sets `nCnt` blocks starting at `pDst` to `nPattern` (block-size is given by the type of `pDst`).	
Particularities and Limitations	
> This macro is synchronous. > This macro is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context..	
Call context	
> ANY	

Table 4-5 VStdLib_MemSetMacro

4.2.3 Memory copy

4.2.3.1 VStdLib_MemCpy

Prototype	
`void VStdLib_MemCpy ( void *pDst, const void *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Copies data from one memory location to another. Copies `nCnt` bytes starting at `pSrc` to another memory location starting at `pDst`.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`. The compatible definition `VStdLib_MemCpyLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-6 VStdLib_MemCpy

4.2.3.2 VStdLib_MemCpy16

Prototype	
`void VStdLib_MemCpy16 ( uint16 *pDst, const uint16 *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, 16-bit aligned and not NULL.
`pSrc [in]`	Pointer to the memory location to copy from, 16-bit aligned and not NULL.
`nCnt [in]`	Number of 16-bit blocks to copy, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Copies data from one memory location to another. Copies `nCnt` 16-bit blocks starting at `pSrc` to another memory location starting at `pDst`.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`. The compatible definition VStdLib_`MemCpy16Large()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-7 VStdLib_MemCpy16

4.2.3.3 VStdLib_MemCpy32

Prototype	
`void VStdLib_MemCpy32 ( uint32 *pDst, const uint32 *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, 32-bit aligned and not NULL.
`pSrc [in]`	Pointer to the memory location to copy from, 32-bit aligned and not NULL.
`nCnt [in]`	Number of 32-bit blocks to copy, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Copies data from one memory location to another. Copies `nCnt` 32-bit blocks starting at `pSrc` to another memory location starting at `pDst`.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`. The compatible definition `VStdLib_MemCpy32Large()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-8 VStdLib_MemCpy32

4.2.3.4 VStdLib_MemCpy_s

Prototype	
`void VStdLib_MemCpy_s ( void *pDst, VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy.
Return code	
void	none
Functional Description	
Verifies the destination size and copies data from one memory location to another. Uses `VStdLib_MemCpy()` to copy `nCnt` bytes starting at `pSrc` to another memory location starting at `pDst`, if `nDstSize` is greater than or equal to `nCnt`.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nDstSize`. > The parameters `pDst` and `nDstSize` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): The compatible definition `VStdLib_MemCpyLarge_s()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-9 VStdLib_MemCpy_s

4.2.3.5 VStdLib_MemCpyMacro

Prototype	
`void VStdLib_MemCpyMacro ( AnyPtrType *pDst, AnyPtrType *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Any typed pointer to the memory location to copy to, must be aligned corresponding to its type and not be NULL.
`pSrc [in]`	Any typed pointer to the memory location to copy from, must be aligned corresponding to its type and not be NULL.
`nCnt [in]`	Number of blocks to copy, `pDst` must be valid for this amount.
Return code	
`void`	none
Functional Description	
Copies data from one memory location to another (macro implementation). Copies `nCnt` blocks starting at `pSrc` to another memory location starting at `pDst` (block-size is given by the type of `pDst`).	
Particularities and Limitations	
> This macro is synchronous. > This macro is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed.	
Call context	
> ANY	

Table 4-10 VStdLib_MemCpyMacro

4.2.3.6 VStdLib_MemCpyMacro_s

Prototype	
`void VStdLib_MemCpyMacro_s ( AnyPtrType *pDst, VStdLib_CntType nDstSize, AnyPtrType *pSrc, VStdLib_CntType nCnt )`	
Parameter	
`pDst [out]`	Any typed pointer to the memory location to copy to, must be aligned corresponding to its type and not be NULL.
`nDstSize [in]`	Maximum number of blocks available in the destination (typically the size of the destination object).
`pSrc [in]`	Any typed pointer to the memory location to copy from, must be aligned corresponding to its type and not be NULL.
`nCnt [in]`	Number of blocks to copy.
Return code	
`void`	none
Functional Description	
Verifies the destination size and copies data from one memory location to another (macro implementation). Uses `VStdLib_MemCpyMacro()` to copy `nCnt` blocks starting at `pSrc` to another memory location starting at `pDst` (block-size is given by the type of `pDst`), if `nDstSize` is greater than or equal to `nCnt`.	
Particularities and Limitations	
> This macro is synchronous. > This macro is reentrant for different memory areas defined by parameters `pDst` and `nDstSize`. > The parameters `pDst` and `nDstSize` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed.	
Call context	
> ANY	

Table 4-11 VStdLib_MemCpyMacro_s

4.2.4 Memory copy with endianness conversion

4.2.4.1 VStdLib_MemCpySwapEndian

Prototype	
`void VStdLib_MemCpySwapEndian ( void *pDst, const void *pSrc, VStdLib_CntType nCnt, uint8 intType )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
Return code	
`void`	none
Functional Description	
Copies `nCnt` bytes starting at `pSrc` to another memory location starting at `pDst` in the swapped endianness according to the integer type `intType`. Depending on the configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt` different memory access sizes are used (see TechRef). `intType` value `VSTDLIB_INTTYPE_64` is only supported if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`) and writable from the calling context. > The parameter `pDst` is aligned according to the integer type `intType`. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_MEM_CPY_ENDIAN_CONVERSION_LIBRARY_FUNCTION == STD_ON`. The compatible definition `VStdLib_MemCpySwapEndianLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-12 VStdLib_MemCpySwapEndian

4.2.4.2 VStdLib_MemCpySwapEndian_s

Prototype
`void` **`VStdLib_MemCpySwapEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt, uint8 intType, Std_ReturnType errorCode )`

Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
`errorCode [in]`	Value to be returned, if `nCnt` exceeds `nDstSize`. Should not be equal to `E_OK`.

Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` exceeds `nDstSize`. No copying was performed.

Functional Description

Verifies the destination size and copies data from one memory location to another in the swapped endianness according to the integer type `intType`.

If `nDstSize` is greater than or equal to `nCnt`, it uses `VStdLib_MemCpySwapEndian()` to copy and swap `nCnt` bytes starting at `pSrc` to another memory location starting at `pDst`. Otherwise no copying is performed and the passed `errorCode` is returned.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters `pDst` and `nDstSize`.
- The parameters `pDst` and `nDstSize` have to define a valid memory area that is writable from the calling context.
- The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`).
- The parameter `pDst` is aligned according to the integer type `intType`.
- The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed.
- The parameter `errorCode` must not be `E_OK`.
- Configuration Variant(s): The compatible definition `VStdLib_MemCpySwapEndianLarge_s()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.

Call context
> ANY

Table 4-13 VStdLib_MemCpySwapEndian_s

4.2.4.3 VStdLib_MemCpyEndianPlatformToBig

Prototype
`void VStdLib_MemCpyEndianPlatformToBig ( void *pDst, const void *pSrc, VStdLib_CntType nCnt, uint8 intType )`

Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).

Return code	
`void`	none

Functional Description

Copies `nCnt` bytes starting at `pSrc` (platform endianness) to another memory location starting at `pDst` in big endian according to the integer type `intType`.

Depending on the configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt` different memory access sizes are used (see TechRef).

`intType` value `VSTDLIB_INTTYPE_64` is only supported if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`.
- The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`) and writable from the calling context.
- The parameter `pDst` is aligned according to the integer type `intType`.
- The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed.
- Configuration Variant(s): The compatible definition `VStdLib_MemCpyEndianPlatformToBigLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.

Call context
> ANY

Table 4-14 VStdLib_MemCpyEndianPlatformToBig

4.2.4.4 VStdLib_MemCpyEndianPlatformToBig_s

Prototype

```
void VStdLib_MemCpyEndianPlatformToBig_s ( void *pDst,
VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt,
uint8 intType, Std_ReturnType errorCode )
```

Parameter

pDst [out]	Pointer to the memory location to copy to, must not be NULL.
nDstSize [in]	Maximum number of bytes available in the destination (typically the size of the destination object).
pSrc [in]	Pointer to the memory location to copy from, must not be NULL.
nCnt [in]	Number of bytes to copy, pDst must be valid for this amount.
intType [in]	Integer type with which the memory locations are interpreted (VSTDLIB_INTTYPE_[8, 16, 32, 64]).
errorCode [in]	Value to be returned, if nCnt exceeds nDstSize. Should not be equal to E_OK.

Return code

E_OK	Copying was performed.
errorCode	nCnt exceeds nDstSize. No copying was performed.

Functional Description

Verifies the destination size and copies data from one memory location (platform endianness) to another in big endian according to the integer type intType.

If nDstSize is greater than or equal to nCnt, it uses VStdLib_MemCpy() / VStdLib_MemCpySwapEndian() to copy nCnt bytes starting at pSrc to another memory location starting at pDst in big endian. Otherwise no copying is performed and the passed errorCode is returned.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters pDst and nDstSize.
- The parameters pDst and nDstSize have to define a valid memory area that is writable from the calling context.
- The parameters pDst and nCnt have to define a valid memory area that is valid for a multiple of the integer type size (parameter intType).
- The parameter pDst is aligned according to the integer type intType.
- The source memory area defined by parameters pSrc and nCnt must not be changed by another context while the copying is being executed.
- The parameter errorCode must not be E_OK.
- Configuration Variant(s): The compatible definition VStdLib_MemCpyEndianPlatformToBigLarge_s() exists additionally (and solely) if VSTDLIB_SUPPORT_LARGE_DATA == STD_ON.

Call context
> ANY

Table 4-15 VStdLib_MemCpyEndianPlatformToBig_s

4.2.4.5 VStdLib_MemCpyEndianPlatformToLittle

Prototype	
`void VStdLib_MemCpyEndianPlatformToLittle ( void *pDst, const void *pSrc, VStdLib_CntType nCnt, uint8 intType )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
Return code	
`void`	none
Functional Description	
Copies `nCnt` bytes starting at `pSrc` (platform endianness) to another memory location starting at `pDst` in little endian according to the integer type `intType`. Depending on the configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt` different memory access sizes are used (see TechRef). `intType` value `VSTDLIB_INTTYPE_64` is only supported if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`) and writable from the calling context. > The parameter `pDst` is aligned according to the integer type `intType`. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): The compatible definition `VStdLib_MemCpyEndianPlatformToLittleLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-16 VStdLib_MemCpyEndianPlatformToLittle

4.2.4.6 VStdLib_MemCpyEndianPlatformToLittle_s

Prototype

```
void VStdLib_MemCpyEndianPlatformToLittle_s ( void *pDst,
VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt,
uint8 intType, Std_ReturnType errorCode )
```

Parameter

pDst [out]	Pointer to the memory location to copy to, must not be NULL.
nDstSize [in]	Maximum number of bytes available in the destination (typically the size of the destination object).
pSrc [in]	Pointer to the memory location to copy from, must not be NULL.
nCnt [in]	Number of bytes to copy, pDst must be valid for this amount.
intType [in]	Integer type with which the memory locations are interpreted (VSTDLIB_INTTYPE_[8, 16, 32, 64]).
errorCode [in]	Value to be returned, if nCnt exceeds nDstSize. Should not be equal to E_OK.

Return code

E_OK	Copying was performed.
errorCode	nCnt exceeds nDstSize. No copying was performed.

Functional Description

Verifies the destination size and copies data from one memory location (platform endianness) to another in little endian according to the integer type intType.

If nDstSize is greater than or equal to nCnt, it uses VStdLib_MemCpy() / VStdLib_MemCpySwapEndian() to copy nCnt bytes starting at pSrc to another memory location starting at pDst in little endian. Otherwise no copying is performed and the passed errorCode is returned.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters pDst and nDstSize.
- The parameters pDst and nDstSize have to define a valid memory area that is writable from the calling context.
- The parameters pDst and nCnt have to define a valid memory area that is valid for a multiple of the integer type size (parameter intType).
- The parameter pDst is aligned according to the integer type intType.
- The source memory area defined by parameters pSrc and nCnt must not be changed by another context while the copying is being executed.
- The parameter errorCode must not be E_OK.
- Configuration Variant(s): The compatible definition VStdLib_MemCpyEndianPlatformToLittleLarge_s() exists additionally (and solely) if VSTDLIB_SUPPORT_LARGE_DATA == STD_ON.

Call context
> ANY

Table 4-17 VStdLib_MemCpyEndianPlatformToLittle_s

4.2.4.7 VStdLib_MemCpyEndianBigToPlatform

Prototype	
`void VStdLib_MemCpyEndianBigToPlatform ( void *pDst, const void *pSrc, VStdLib_CntType nCnt, uint8 intType )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
Return code	
`void`	none
Functional Description	
Copies `nCnt` bytes starting at `pSrc` (big endian) to another memory location starting at `pDst` in the platform endianness according to the integer type `intType`. Depending on the configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt` different memory access sizes are used (see TechRef). `intType` value `VSTDLIB_INTTYPE_64` is only supported if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`) and writable from the calling context. > The parameter `pDst` is aligned according to the integer type `intType`. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): The compatible definition `VStdLib_MemCpyEndianBigToPlatformLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-18 VStdLib_MemCpyEndianBigToPlatform

4.2.4.8 VStdLib_MemCpyEndianBigToPlatform_s

Prototype

```
void VStdLib_MemCpyEndianBigToPlatform_s ( void *pDst,
VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt,
uint8 intType, Std_ReturnType errorCode )
```

Parameter

pDst [out]	Pointer to the memory location to copy to, must not be NULL.
nDstSize [in]	Maximum number of bytes available in the destination (typically the size of the destination object).
pSrc [in]	Pointer to the memory location to copy from, must not be NULL.
nCnt [in]	Number of bytes to copy, pDst must be valid for this amount.
intType [in]	Integer type with which the memory locations are interpreted (VSTDLIB_INTTYPE_[8, 16, 32, 64]).
errorCode [in]	Value to be returned, if nCnt exceeds nDstSize. Should not be equal to E_OK.

Return code

E_OK	Copying was performed.
errorCode	nCnt exceeds nDstSize. No copying was performed.

Functional Description

Verifies the destination size and copies data from one memory location (big endian) to another in the platform endianness according to the integer type intType.

If nDstSize is greater than or equal to nCnt, it uses VStdLib_MemCpy() / VStdLib_MemCpySwapEndian() to copy nCnt bytes starting at pSrc to another memory location starting at pDst in the platform endianness. Otherwise no copying is performed and the passed errorCode is returned.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters pDst and nDstSize.
- The parameters pDst and nDstSize have to define a valid memory area that is writable from the calling context.
- The parameters pDst and nCnt have to define a valid memory area that is valid for a multiple of the integer type size (parameter intType).
- The parameter pDst is aligned according to the integer type intType.
- The source memory area defined by parameters pSrc and nCnt must not be changed by another context while the copying is being executed.
- The parameter errorCode must not be E_OK.
- Configuration Variant(s): The compatible definition VStdLib_MemCpyEndianBigToPlatformLarge_s() exists additionally (and solely) if VSTDLIB_SUPPORT_LARGE_DATA == STD_ON.

Call context
> ANY

Table 4-19 VStdLib_MemCpyEndianBigToPlatform_s

4.2.4.9 VStdLib_MemCpyEndianLittleToPlatform

Prototype	
`void VStdLib_MemCpyEndianLittleToPlatform ( void *pDst, const void *pSrc, VStdLib_CntType nCnt, uint8 intType )`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
Return code	
`void`	none
Functional Description	
Copies `nCnt` bytes starting at `pSrc` (little endian) to another memory location starting at `pDst` in the platform endianness according to the integer type `intType`. Depending on the configuration (`VSTDLIB_RUNTIME_OPTIMIZATION`) and the passed arguments for parameter `intType` and `nCnt` different memory access sizes are used (see TechRef). `intType` value `VSTDLIB_INTTYPE_64` is only supported if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst` and `nCnt`. > The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`) and writable from the calling context. > The parameter `pDst` is aligned according to the integer type `intType`. > The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): The compatible definition `VStdLib_MemCpyEndianLittleToPlatformLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
> ANY	

Table 4-20 VStdLib_MemCpyEndianLittleToPlatform

4.2.4.10 VStdLib_MemCpyEndianLittleToPlatform_s

Prototype
`void VStdLib_MemCpyEndianLittleToPlatform_s ( void *pDst, VStdLib_CntType nDstSize, const void *pSrc, VStdLib_CntType nCnt, uint8 intType, Std_ReturnType errorCode )`

Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nCnt [in]`	Number of bytes to copy, `pDst` must be valid for this amount.
`intType [in]`	Integer type with which the memory locations are interpreted (`VSTDLIB_INTTYPE_[8, 16, 32, 64]`).
`errorCode [in]`	Value to be returned, if `nCnt` exceeds `nDstSize`. Should not be equal to `E_OK`.

Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` exceeds `nDstSize`. No copying was performed.

Functional Description

Verifies the destination size and copies data from one memory location (little endian) to another in the platform endianness according to the integer type `intType`.

If `nDstSize` is greater than or equal to `nCnt`, it uses `VStdLib_MemCpy()` / `VStdLib_MemCpySwapEndian()` to copy `nCnt` bytes starting at `pSrc` to another memory location starting at `pDst` in the platform endianness. Otherwise no copying is performed and the passed `errorCode` is returned.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters `pDst` and `nDstSize`.
- The parameters `pDst` and `nDstSize` have to define a valid memory area that is writable from the calling context.
- The parameters `pDst` and `nCnt` have to define a valid memory area that is valid for a multiple of the integer type size (parameter `intType`).
- The parameter `pDst` is aligned according to the integer type `intType`.
- The source memory area defined by parameters `pSrc` and `nCnt` must not be changed by another context while the copying is being executed.
- Parameter `errorCode` must not be `E_OK`.
- Configuration Variant(s): The compatible definition `VStdLib_MemCpyEndianLittleToPlatformLarge_s()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.

Call context
> ANY

Table 4-21 VStdLib_MemCpyEndianLittleToPlatform_s

4.2.5 Bit copy

4.2.5.1 VStdLib_BitCpySawBigToBigEndian

Prototype	
`void VStdLib_BitCpySawBigToBigEndian ( void *pDst, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of big endian data from one memory location to another memory location with big endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'	
> This function is synchronous.	
> This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`.	
> The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context.	
> The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-22 VStdLib_BitCpySawBigToBigEndian

4.2.5.2 VStdLib_BitCpySawBigToLittleEndian

Prototype	
`void VStdLib_BitCpySawBigToLittleEndian ( void *pDst, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of big endian data from one memory location to another memory location with little endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-23 VStdLib_BitCpySawBigToLittleEndian

4.2.5.3 VStdLib_BitCpySawLittleToBigEndian

Prototype	
`void VStdLib_BitCpySawLittleToBigEndian ( void *pDst, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of little endian data from one memory location to another memory location with big endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-24 VStdLib_BitCpySawLittleToBigEndian

4.2.5.4 VStdLib_BitCpySawLittleToLittleEndian

Prototype	
`void` **`VStdLib_BitCpySawLittleToLittleEndian`** `( void *pDst, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of little endian data from one memory location to another memory location with little endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-25 VStdLib_BitCpySawLittleToLittleEndian

4.2.5.5 VStdLib_BitCpySawBigToBigEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpySawBigToBigEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of big endian data from one memory location to another memory location with big endian data. Uses `VStdLib_BitCpySawBigToBigEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-26 VStdLib_BitCpySawBigToBigEndian_s

4.2.5.6 VStdLib_BitCpySawBigToLittleEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpySawBigToLittleEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of big endian data from one memory location to another memory location with little endian data. Uses `VStdLib_BitCpySawBigToLittleEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-27 VStdLib_BitCpySawBigToLittleEndian_s

4.2.5.7 VStdLib_BitCpySawLittleToBigEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpySawLittleToBigEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of little endian data from one memory location to another memory location with big endian data. Uses `VStdLib_BitCpySawLittleToBigEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-28 VStdLib_BitCpySawLittleToBigEndian_s

4.2.5.8 VStdLib_BitCpySawLittleToLittleEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpySawLittleToLittleEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetSaw, const void *pSrc, VStdLib_CntType nSrcBitOffsetSaw, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetSaw [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetSaw`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetSaw` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of little endian data from one memory location to another memory location with little endian data. Uses `VStdLib_BitCpySawLittleToLittleEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffetSaw` and `nCnt`. > The parameters `pDst`, `nDstBitOffetSaw` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffetSaw` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-29 VStdLib_BitCpySawLittleToLittleEndian_s

4.2.5.9 VStdLib_BitCpyMonBigToBigEndian

Prototype	
`void VStdLib_BitCpyMonBigToBigEndian ( void *pDst, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of big endian data from one memory location to another memory location with big endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-30 VStdLib_BitCpyMonBigToBigEndian

4.2.5.10 VStdLib_BitCpyMonBigToLittleEndian

Prototype	
`void VStdLib_BitCpyMonBigToLittleEndian ( void *pDst, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of big endian data from one memory location to another memory location with little endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-31 VStdLib_BitCpyMonBigToLittleEndian

4.2.5.11 VStdLib_BitCpyMonLittleToBigEndian

Prototype	
`void VStdLib_BitCpyMonLittleToBigEndian ( void *pDst, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
Return code	
`void`	none
Functional Description	
Copies `nCnt` bits of little endian data from one memory location to another memory location with big endian data.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
> ANY	

Table 4-32 VStdLib_BitCpyMonLittleToBigEndian

4.2.5.12 VStdLib_BitCpyMonLittleToLittleEndian

Prototype
`void` **`VStdLib_BitCpyMonLittleToLittleEndian`** `( void *pDst, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt)`

Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in monotone numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.

Return code	
`void`	none

Functional Description

Copies `nCnt` bits of little endian data from one memory location to another memory location with little endian data.

Particularities and Limitations

- Service ID: see table 'Service IDs'
- This function is synchronous.
- This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`.
- The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context.
- The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed.
- Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS == STD_ON`.

Call context

- ANY

Table 4-33 VStdLib_BitCpyMonLittleToLittleEndian

4.2.5.13 VStdLib_BitCpyMonBigToBigEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpyMonBigToBigEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of big endian data from one memory location to another memory location with big endian data. Uses `VStdLib_BitCpyMonBigToBigEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-34 VStdLib_BitCpyMonBigToBigEndian_s

4.2.5.14 VStdLib_BitCpyMonBigToLittleEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpyMonBigToLittleEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of big endian data from one memory location to another memory location with little endian data. Uses `VStdLib_BitCpyMonBigToLittleEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-35 VStdLib_BitCpyMonBigToLittleEndian_s

4.2.5.15 VStdLib_BitCpyMonLittleToBigEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpyMonLittleToBigEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of little endian data from one memory location to another memory location with big endian data. Uses `VStdLib_BitCpyMonLittleToBigEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-36 VStdLib_BitCpyMonLittleToBigEndian_s

4.2.5.16 VStdLib_BitCpyMonLittleToLittleEndian_s

Prototype	
`Std_ReturnType` **`VStdLib_BitCpyMonLittleToLittleEndian_s`** `( void *pDst, VStdLib_CntType nDstSize, VStdLib_CntType nDstBitOffsetMon, const void *pSrc, VStdLib_CntType nSrcBitOffsetMon, VStdLib_CntType nCnt, Std_ReturnType errorCode)`	
Parameter	
`pDst [out]`	Pointer to the memory location to copy to, must not be NULL.
`nDstSize [in]`	Maximum number of bytes available in the destination (typically the size of the destination object).
`nDstBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pDst`. Bits are copied to exactly this bit position.
`pSrc [in]`	Pointer to the memory location to copy from, must not be NULL.
`nSrcBitOffsetMon [in]`	Bit offset (it can even be greater than 7) in sawtooth numbering is added to `pSrc`. Bits are copied from exactly this bit position.
`nCnt [in]`	Number of bits to copy starting at `nDstBitOffsetMon`.
`errorCode [in]`	Value to be returned, if `nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. Should not be equal to `E_OK`.
Return code	
`E_OK`	Copying was performed.
`errorCode`	`nCnt` bits starting at `nDstBitOffsetMon` exceeds `nDstSize`. No copying was performed.
Functional Description	
Verifies the destination size and copies `nCnt` bits of little endian data from one memory location to another memory location with little endian data. Uses `VStdLib_BitCpyMonLittleToLittleEndian()` to copy data, if `nDstSize` is greater than or equal to the last byte to copy to.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameters `pDst`, `nDstBitOffsetMon` and `nCnt`. > The parameters `pDst`, `nDstBitOffsetMon` and `nCnt` have to define a valid memory area that is writable from the calling context. > The source memory area defined by parameters `pSrc`, `nSrcBitOffsetMon` and `nCnt` must not be changed by another context while the copying is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): none	
Call context	
> ANY	

Table 4-37 VStdLib_BitCpyMonLittleToLittleEndian_s

4.2.6 Memory comparison

4.2.6.1 VStdLib_MemCmp

Prototype	
`sint8 VStdLib_MemCmp ( const void *pBuf1, const void *pBuf2, VStdLib_CntType nCnt )`	
Parameter	
`pBuf1 [in]`	Pointer to first buffer for comparison, must not be NULL.
`pBuf2 [in]`	Pointer to second buffer for comparison, must not be NULL.
`nCnt [in]`	Number of bytes to compare, `pBuf1` and `pBuf2` must be valid for this amount.
Return code	
`< 0`	The first buffer contains a smaller value than the second buffer.
`0`	The two buffers are equal for the number of compared bytes.
`> 0`	The first buffer contains a greater value than the second buffer.
Functional Description	
Compares two buffers for the specified number of bytes and returns whether they are equal or not.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant. > The parameters `pBuf1` and `pBuf2` have to point to valid memory areas. Each memory area has to be valid for at least the byte count defined in parameter `nCnt`. > The memory areas defined by parameters `pBuf1`, `pBuf2` and `nCnt` must not be changed by another context while the comparison is being executed. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`. The compatible definition `VStdLib_MemCmpLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
ANY	

Table 4-38 VStdLib_MemCmp

4.2.6.2 VStdLib_MemCheck

Prototype	
`Std_ReturnType` **`VStdLib_MemCheck`** `( const void *pBuf, uint8 nPattern, VStdLib_CntType nCnt, Std_ReturnType errorCode )`	
Parameter	
`pBuf [in]`	Pointer to the buffer to be checked, must not be NULL.
`nPattern [in]`	The character to be checked for in the buffer.
`nCnt [in]`	Number of bytes to check, `pBuf` must be valid for this amount.
`errorCode [in]`	Value to be returned, if `pBuf` consists not only of pattern. Should not be equal to `E_OK`.
Return code	
`E_OK`	`pBuf` consists only of the nPattern.
`errorCode`	`pBuf` contains at least one value that is not equal to nPattern.
Functional Description	
Compares a memory area to a specified pattern and returns whether the memory area consists only of the pattern or not.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant. > The parameters `pBuf` must point to a valid memory area. The memory area must be valid for at least the byte count defined in parameter `nCnt`. > The memory area defined by parameters `pBuf` and `nCnt` must not be changed by another context while the comparison is being executed. > The parameter `errorCode` must not be `E_OK`. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_MEM_CHECK_LIBRARY_FUNCTION == STD_ON`. The compatible definition `VStdLib_MemCHeckLarge()` exists additionally (and solely) if `VSTDLIB_SUPPORT_LARGE_DATA == STD_ON`.	
Call context	
ANY	

Table 4-39 VStdLib_MemCheck

4.2.7 Bit search

4.2.7.1 VStdLib_GetHighestBitPosOne8

Prototype	
`uint8 VStdLib_GetHighestBitPosOne8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the highest set bit shall be found.
Return code	
`0-7`	Bit position of the highest set bit.
`VSTDLIB_BITPOS_NA`	8-bit value contains no set bit.
Functional Description	
Returns the highest bit position of a set bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-40 VStdLib_GetHighestBitPosOne8

4.2.7.2 VStdLib_GetHighestBitPosOne16

Prototype	
`uint8 VStdLib_GetHighestBitPosOne16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the highest set bit shall be found.
Return code	
`0-15`	Bit position of the highest set bit.
`VSTDLIB_BITPOS_NA`	16-bit value contains no set bit.
Functional Description	
Returns the highest bit position of a set bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-41 VStdLib_GetHighestBitPosOne16

4.2.7.3 VStdLib_GetHighestBitPosOne32

Prototype	
`uint8 VStdLib_GetHighestBitPosOne32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the highest set bit shall be found.
Return code	
`0-31`	Bit position of the highest set bit.
`VSTDLIB_BITPOS_NA`	32-bit value contains no set bit.
Functional Description	
Returns the highest bit position of a set bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-42 VStdLib_GetHighestBitPosOne32

4.2.7.4 VStdLib_GetHighestBitPosOne64

Prototype	
`uint8 VStdLib_GetHighestBitPosOne64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the highest set bit shall be found.
Return code	
`0-63`	Bit position of the highest set bit.
`VSTDLIB_BITPOS_NA`	64-bit value contains no set bit.
Functional Description	
Returns the highest bit position of a set bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-43 VStdLib_GetHighestBitPosOne64

4.2.7.5 VStdLib_GetHighestBitPosZero8

Prototype	
`uint8 VStdLib_GetHighestBitPosZero8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the highest unset bit shall be found.
Return code	
`0-7`	Bit position of the highest unset bit.
`VSTDLIB_BITPOS_NA`	8-bit value contains no unset bit.
Functional Description	
Returns the highest bit position of an unset bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-44 VStdLib_GetHighestBitPosZero8

4.2.7.6 VStdLib_GetHighestBitPosZero16

Prototype	
`uint8 VStdLib_GetHighestBitPosZero16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the highest unset bit shall be found.
Return code	
`0-15`	Bit position of the highest unset bit.
`VSTDLIB_BITPOS_NA`	16-bit value contains no unset bit.
Functional Description	
Returns the highest bit position of an unset bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-45 VStdLib_GetHighestBitPosZero16

4.2.7.7 VStdLib_GetHighestBitPosZero32

Prototype	
`uint8 VStdLib_GetHighestBitPosZero32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the highest unset bit shall be found.
Return code	
`0-31`	Bit position of the highest unset bit.
`VSTDLIB_BITPOS_NA`	32-bit value contains no unset bit.
Functional Description	
Returns the highest bit position of an unset bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-46 VStdLib_GetHighestBitPosZero32

4.2.7.8 VStdLib_GetHighestBitPosZero64

Prototype	
`uint8 VStdLib_GetHighestBitPosZero64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the highest unset bit shall be found.
Return code	
`0-63`	Bit position of the highest unset bit.
`VSTDLIB_BITPOS_NA`	64-bit value contains no unset bit.
Functional Description	
Returns the highest bit position of an unset bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-47 VStdLib_GetHighestBitPosZero8

4.2.7.9 VStdLib_GetLowestBitPosOne8

Prototype	
`uint8 VStdLib_GetLowestBitPosOne8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the lowest set bit shall be found.
Return code	
`0-7`	Bit position of the lowest set bit.
`VSTDLIB_BITPOS_NA`	8-bit value contains no set bit.
Functional Description	
Returns the lowest bit position of a set bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-48 VStdLib_GetLowestBitPosOne8

4.2.7.10 VStdLib_GetLowestBitPosOne16

Prototype	
`uint8 VStdLib_GetLowestBitPosOne16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the lowest set bit shall be found.
Return code	
`0-15`	Bit position of the lowest set bit.
`VSTDLIB_BITPOS_NA`	16-bit value contains no set bit.
Functional Description	
Returns the lowest bit position of a set bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-49 VStdLib_GetLowestBitPosOne16

4.2.7.11 VStdLib_GetLowestBitPosOne32

Prototype	
`uint8 VStdLib_GetLowestBitPosOne32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the lowest set bit shall be found.
Return code	
`0-31`	Bit position of the lowest set bit.
`VSTDLIB_BITPOS_NA`	32-bit value contains no set bit.
Functional Description	
Returns the lowest bit position of a set bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-50 VStdLib_GetLowestBitPosOne32

4.2.7.12 VStdLib_GetLowestBitPosOne64

Prototype	
`uint8 VStdLib_GetLowestBitPosOne64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the lowest set bit shall be found.
Return code	
`0-63`	Bit position of the lowest set bit.
`VSTDLIB_BITPOS_NA`	64-bit value contains no set bit.
Functional Description	
Returns the lowest bit position of a set bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-51 VStdLib_GetLowestBitPosOne64

4.2.7.13 VStdLib_GetLowestBitPosZero8

Prototype	
`uint8 VStdLib_GetLowestBitPosZero8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the lowest unset bit shall be found.
Return code	
`0-7`	Bit position of the lowest unset bit.
`VSTDLIB_BITPOS_NA`	8-bit value contains no unset bit.
Functional Description	
Returns the lowest bit position of an unset bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-52 VStdLib_GetLowestBitPosZero8

4.2.7.14 VStdLib_GetLowestBitPosZero16

Prototype	
`uint8 VStdLib_GetLowestBitPosZero16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the lowest unset bit shall be found.
Return code	
`0-15`	Bit position of the lowest unset bit.
`VSTDLIB_BITPOS_NA`	16-bit value contains no unset bit.
Functional Description	
Returns the lowest bit position of an unset bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-53 VStdLib_GetLowestBitPosZero16

4.2.7.15 VStdLib_GetLowestBitPosZero32

Prototype	
`uint8 VStdLib_GetLowestBitPosZero32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the lowest unset bit shall be found.
Return code	
`0-31`	Bit position of the lowest unset bit.
`VSTDLIB_BITPOS_NA`	32-bit value contains no unset bit.
Functional Description	
Returns the lowest bit position of an unset bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-54 VStdLib_GetLowestBitPosZero32

4.2.7.16 VStdLib_GetLowestBitPosZero64

Prototype	
`uint8 VStdLib_GetLowestBitPosZero64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the lowest unset bit shall be found.
Return code	
`0-63`	Bit position of the lowest unset bit.
`VSTDLIB_BITPOS_NA`	64-bit value contains no unset bit.
Functional Description	
Returns the lowest bit position of an unset bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-55 VStdLib_GetLowestBitPosZero64

4.2.7.17 VStdLib_GetCountLeadingOnes8

Prototype	
`uint8 VStdLib_GetCountLeadingOnes8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the count of leading ones shall be determined.
Return code	
`0-7`	Count of leading ones.
`VSTDLIB_BITCNT_NA`	8-bit value contains no unset bit.
Functional Description	
Returns the count of set bits preceding the highest unset bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-56 VStdLib_GetCountLeadingOnes8

4.2.7.18 VStdLib_GetCountLeadingOnes16

Prototype	
`uint8 VStdLib_GetCountLeadingOnes16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the count of leading ones shall be determined.
Return code	
`0-15`	Count of leading ones.
`VSTDLIB_BITCNT_NA`	16-bit value contains no unset bit.
Functional Description	
Returns the count of set bits preceding the highest unset bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-57 VStdLib_GetCountLeadingOnes16

4.2.7.19 VStdLib_GetCountLeadingOnes32

Prototype	
`uint8 VStdLib_GetCountLeadingOnes32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the count of leading ones shall be determined.
Return code	
`0-31`	Count of leading ones.
`VSTDLIB_BITCNT_NA`	32-bit value contains no unset bit.
Functional Description	
Returns the count of set bits preceding the highest unset bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-58 VStdLib_GetCountLeadingOnes32

4.2.7.20 VStdLib_GetCountLeadingOnes64

Prototype	
`uint8 VStdLib_GetCountLeadingOnes64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the count of leading ones shall be determined.
Return code	
`0-63`	Count of leading ones.
`VSTDLIB_BITCNT_NA`	64-bit value contains no unset bit.
Functional Description	
Returns the count of set bits preceding the highest unset bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-59 VStdLib_GetCountLeadingOnes64

4.2.7.21 VStdLib_GetCountLeadingZeros8

Prototype	
`uint8 VStdLib_GetCountLeadingZeros8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the count of leading zeros shall be determined.
Return code	
`0-7`	Count of leading zeros.
`VSTDLIB_BITCNT_NA`	8-bit value contains no set bit.
Functional Description	
Returns the count of unset bits preceding the highest set bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-60 VStdLib_GetCountLeadingZeros8

4.2.7.22 VStdLib_GetCountLeadingZeros16

Prototype	
`uint8 VStdLib_GetCountLeadingZeros16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the count of leading zeros shall be determined.
Return code	
`0-15`	Count of leading zeros.
`VSTDLIB_BITCNT_NA`	16-bit value contains no set bit.
Functional Description	
Returns the count of unset bits preceding the highest set bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-61 VStdLib_GetCountLeadingZeros16

4.2.7.23 VStdLib_GetCountLeadingZeros32

Prototype	
`uint8 VStdLib_GetCountLeadingZeros32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the count of leading zeros shall be determined.
Return code	
`0-31`	Count of leading zeros.
`VSTDLIB_BITCNT_NA`	32-bit value contains no set bit.
Functional Description	
Returns the count of unset bits preceding the highest set bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-62 VStdLib_GetCountLeadingZeros32

4.2.7.24 VStdLib_GetCountLeadingZeros64

Prototype	
`uint8 VStdLib_GetCountLeadingZeros64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the count of leading zeros shall be determined.
Return code	
`0-63`	Count of leading zeros.
`VSTDLIB_BITCNT_NA`	64-bit value contains no set bit.
Functional Description	
Returns the count of unset bits preceding the highest set bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-63 VStdLib_GetCountLeadingZeros64

4.2.7.25 VStdLib_GetCountTrailingOnes8

Prototype	
`uint8 VStdLib_GetCountTrailingOnes8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the count of trailing ones shall be determined.
Return code	
`0-7`	Count of trailing ones.
`VSTDLIB_BITCNT_NA`	8-bit value contains no unset bit.
Functional Description	
Returns the count of set bits following the lowest unset bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-64 VStdLib_GetCountTrailingOnes8

4.2.7.26 VStdLib_GetCountTrailingOnes16

Prototype	
`uint8 VStdLib_GetCountTrailingOnes16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the count of trailing ones shall be determined.
Return code	
`0-15`	Count of trailing ones.
`VSTDLIB_BITCNT_NA`	16-bit value contains no unset bit.
Functional Description	
Returns the count of set bits following the lowest unset bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-65 VStdLib_GetCountTrailingOnes16

4.2.7.27 VStdLib_GetCountTrailingOnes32

Prototype	
`uint8 VStdLib_GetCountTrailingOnes32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the count of trailing ones shall be determined.
Return code	
`0-31`	Count of trailing ones.
`VSTDLIB_BITCNT_NA`	32-bit value contains no unset bit.
Functional Description	
Returns the count of set bits following the lowest unset bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-66 VStdLib_GetCountTrailingOnes32

4.2.7.28 VStdLib_GetCountTrailingOnes64

Prototype	
`uint8 VStdLib_GetCountTrailingOnes64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the count of trailing ones shall be determined.
Return code	
`0-63`	Count of trailing ones.
`VSTDLIB_BITCNT_NA`	64-bit value contains no unset bit.
Functional Description	
Returns the count of set bits following the lowest unset bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-67 VStdLib_GetCountTrailingOnes64

4.2.7.29 VStdLib_GetCountTrailingZeros8

Prototype	
`uint8 VStdLib_GetCountTrailingZeros8 ( uint8 value )`	
Parameter	
`value [in]`	The 8-bit value in which the count of trailing zeros shall be determined.
Return code	
`0-7`	Count of trailing zeros.
`VSTDLIB_BITCNT_NA`	8-bit value contains no set bit.
Functional Description	
Returns the count of unset bits following the lowest set bit in the specified 8-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-68 VStdLib_GetCountTrailingZeros8

4.2.7.30 VStdLib_GetCountTrailingZeros16

Prototype	
`uint8 VStdLib_GetCountTrailingZeros16 ( uint16 value )`	
Parameter	
`value [in]`	The 16-bit value in which the count of trailing zeros shall be determined.
Return code	
`0-15`	Count of trailing zeros.
`VSTDLIB_BITCNT_NA`	16-bit value contains no set bit.
Functional Description	
Returns the count of unset bits following the lowest set bit in the specified 16-bit value.	
Particularities and Limitations	
> This function is synchronous.	
> This function is reentrant.	
> Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-69 VStdLib_GetCountTrailingZeros16

4.2.7.31 VStdLib_GetCountTrailingZeros32

Prototype	
`uint8 VStdLib_GetCountTrailingZeros32 ( uint32 value )`	
Parameter	
`value [in]`	The 32-bit value in which the count of trailing zeros shall be determined.
Return code	
`0-31`	Count of trailing zeros.
`VSTDLIB_BITCNT_NA`	32-bit value contains no set bit.
Functional Description	
Returns the count of unset bits following the lowest set bit in the specified 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-70 VStdLib_GetCountTrailingZeros32

4.2.7.32 VStdLib_GetCountTrailingZeros64

Prototype	
`uint8 VStdLib_GetCountTrailingZeros64 ( uint64 value )`	
Parameter	
`value [in]`	The 64-bit value in which the count of trailing zeros shall be determined.
Return code	
`0-63`	Count of trailing zeros.
`VSTDLIB_BITCNT_NA`	64-bit value contains no set bit.
Functional Description	
Returns the count of unset bits following the lowest set bit in the specified 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-71 VStdLib_GetCountTrailingZeros64

4.2.8 Serialization

4.2.8.1 VStdLib_ConvertUint16ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertUint16ToUint8ArrayBigEndian`** `( uint16 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 16-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 16-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least two elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-72 VStdLib_ConvertUint16ToUint8ArrayBigEndian

4.2.8.2 VStdLib_ConvertSint16ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertSint16ToUint8ArrayBigEndian`** `( sint16 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 16-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 16-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least two elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-73 VStdLib_ConvertSint16ToUint8ArrayBigEndian

4.2.8.3 VStdLib_ConvertUint16ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertUint16ToUint8ArrayLittleEndian`** `( uint16 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 16-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 16-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least two elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-74 VStdLib_ConvertUint16ToUint8ArrayLittleEndian

4.2.8.4 VStdLib_ConvertSint16ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertSint16ToUint8ArrayLittleEndian`** `( sint16 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 16-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 16-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least two elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-75 VStdLib_ConvertSint16ToUint8ArrayLittleEndian

4.2.8.5 VStdLib_ConvertUint32ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertUint32ToUint8ArrayBigEndian`** `( uint32 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 32-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 32-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least four elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-76 VStdLib_ConvertUint32ToUint8ArrayBigEndian

4.2.8.6 VStdLib_ConvertSint32ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertSint32ToUint8ArrayBigEndian`** `( sint32 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 32-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 32-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least four elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-77 VStdLib_ConvertSint32ToUint8ArrayBigEndian

4.2.8.7 VStdLib_ConvertUint32ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertUint32ToUint8ArrayLittleEndian`** `( uint32 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 32-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 32-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least four elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-78 VStdLib_ConvertUint32ToUint8ArrayLittleEndian

4.2.8.8 VStdLib_ConvertSint32ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertSint32ToUint8ArrayBigEndian`** `( sint32 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 32-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 32-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least four elements that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-79 VStdLib_ConvertSint32ToUint8ArrayLittleEndian

4.2.8.9 VStdLib_ConvertUint64ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertUint64ToUint8ArrayBigEndian`** `( uint64 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 64-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 64-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least eight elements that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-80 VStdLib_ConvertUint64ToUint8ArrayBigEndian

4.2.8.10 VStdLib_ConvertSint64ToUint8ArrayBigEndian

Prototype	
`void` **`VStdLib_ConvertSint64ToUint8ArrayBigEndian`** `( sint64 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 64-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 64-bit value to an unsigned 8-bit array according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least eight elements that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-81 VStdLib_ConvertSint64ToUint8ArrayBigEndian

4.2.8.11 VStdLib_ConvertUint64ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertUint64ToUint8ArrayLittleEndian`** `( uint64 src, uint8 *dst )`	
Parameter	
`src [in]`	The unsigned 64-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 64-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least eight elements that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-82 VStdLib_ConvertUint64ToUint8ArrayLittleEndian

4.2.8.12 VStdLib_ConvertSint64ToUint8ArrayLittleEndian

Prototype	
`void` **`VStdLib_ConvertSint64ToUint8ArrayLittleEndian`** `( sint64 src, uint8 *dst )`	
Parameter	
`src [in]`	The signed 64-bit value, which shall be converted.
`dst [out]`	Pointer to an unsigned 8-bit array, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified signed 64-bit value to an unsigned 8-bit array according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `dst` has to point to an unsigned 8-bit array of at least eight elements that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-83 VStdLib_ConvertSint64ToUint8ArrayLittleEndian

4.2.9 Deserialization

4.2.9.1 VStdLib_ConvertUint8ArrayToUint16BigEndian

Prototype	
`void VStdLib_ConvertUint8ArrayToUint16BigEndian` `( const uint8 *src, uint16 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted..
`dst [out]`	Pointer to an unsigned 16-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 16-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least two elements. > The parameter `dst` has to point to an unsigned 16-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-84 VStdLib_ConvertUint8ArrayToUint16BigEndian

4.2.9.2 VStdLib_ConvertUint8ArrayToSint16BigEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint16BigEndian`** `( const uint8 *src, sint16 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 16-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 16-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least two elements. > The parameter `dst` has to point to a signed 16-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-85 VStdLib_ConvertUint8ArrayToSint16BigEndian

4.2.9.3 VStdLib_ConvertUint8ArrayToUint16LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToUint16LittleEndian`** `( const uint8 *src, uint16 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to an unsigned 16-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 16-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least two elements. > The parameter `dst` has to point to an unsigned 16-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-86 VStdLib_ConvertUint8ArrayToUint16LittleEndian

4.2.9.4 VStdLib_ConvertUint8ArrayToSint16LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint16LittleEndian`** `( const uint8 *src, sint16 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 16-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 16-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least two elements. > The parameter `dst` has to point to a signed 16-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-87 VStdLib_ConvertUint8ArrayToSint16LittleEndian

4.2.9.5 VStdLib_ConvertUint8ArrayToUint32BigEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToUint32BigEndian`** `( const uint8 *src, uint32 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to an unsigned 32-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 32-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least four elements. > The parameter `dst` has to point to an unsigned 32-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-88 VStdLib_ConvertUint8ArrayToUint32BigEndian

4.2.9.6 VStdLib_ConvertUint8ArrayToSint32BigEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint32BigEndian`** `( const uint8 *src, sint32 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 32-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 32-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least four elements. > The parameter `dst` has to point to a signed 32-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-89 VStdLib_ConvertUint8ArrayToSint32BigEndian

4.2.9.7 VStdLib_ConvertUint8ArrayToUint32LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToUint32LittleEndian`** `( const uint8 *src, uint32 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to an unsigned 32-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 32-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least four elements. > The parameter `dst` has to point to an unsigned 32-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-90 VStdLib_ConvertUint8ArrayToUint32LittleEndian

4.2.9.8 VStdLib_ConvertUint8ArrayToSint32LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint32LittleEndian`** `( const uint8 *src, sint32 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 32-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 32-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least four elements. > The parameter `dst` has to point to a signed 32-bit variable that is writable from the calling context. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-91 VStdLib_ConvertUint8ArrayToSint32LittleEndian

4.2.9.9 VStdLib_ConvertUint8ArrayToUint64BigEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToUint64BigEndian`** `( const uint8 *src, uint64 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to an unsigned 64-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 64-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least eight elements. > The parameter `dst` has to point to an unsigned 64-bit variable that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-92 VStdLib_ConvertUint8ArrayToUint64BigEndian

4.2.9.10 VStdLib_ConvertUint8ArrayToSint64BigEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint64BigEndian`** `( const uint8 *src, sint64 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 64-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 64-bit value according to big endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least eight elements. > The parameter `dst` has to point to a signed 64-bit variable that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-93 VStdLib_ConvertUint8ArrayToSint64BigEndian

4.2.9.11 VStdLib_ConvertUint8ArrayToUint64LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToUint64LittleEndian`** `( const uint8 *src, uint64 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to an unsigned 64-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to an unsigned 64-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least eight elements. > The parameter `dst` has to point to an unsigned 64-bit variable that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-94 VStdLib_ConvertUint8ArrayToUint64LittleEndian

4.2.9.12 VStdLib_ConvertUint8ArrayToSint64LittleEndian

Prototype	
`void` **`VStdLib_ConvertUint8ArrayToSint64LittleEndian`** `( const uint8 *src, sint64 *dst )`	
Parameter	
`src [in]`	Pointer to an unsigned 8-bit array, which shall be converted.
`dst [out]`	Pointer to a signed 64-bit variable, where the conversion result will be stored.
Return code	
`void`	none.
Functional Description	
Converts the specified unsigned 8-bit array to a signed 64-bit value according to little endian.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant for different memory areas defined by parameter `dst`. > The parameter `src` has to point to an unsigned 8-bit array of at least eight elements. > The parameter `dst` has to point to a signed 64-bit variable that is writable from the calling context. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-95 VStdLib_ConvertUint8ArrayToSint64LittleEndian

4.2.10 Endianness swap

4.2.10.1 VStdLib_SwapEndianUint16

Prototype	
`uint16` **`VStdLib_SwapEndianUint16`** `( uint16 value )`	
Parameter	
`value [in]`	Unsigned 16-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified unsigned 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-96 VStdLib_SwapEndianUint16

4.2.10.2 VStdLib_SwapEndianSint16

Prototype	
`sint16` **`VStdLib_SwapEndianSint16`** `( sint16 value )`	
Parameter	
`value [in]`	Signed 16-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified signed 16-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_16BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-97 VStdLib_SwapEndianSint16

4.2.10.3 VStdLib_SwapEndianUint32

Prototype	
`uint32 VStdLib_SwapEndianUint32 ( uint32 value )`	
Parameter	
`value [in]`	Unsigned 32-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified unsigned 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-98 VStdLib_SwapEndianUint32

4.2.10.4 VStdLib_SwapEndianSint32

Prototype	
`sint32 VStdLib_SwapEndianSint32 ( sint32 value )`	
Parameter	
`value [in]`	Signed 32-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified signed 32-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is implemented externally if `VSTDLIB_USE_32BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-99 VStdLib_SwapEndianSint32

4.2.10.5 VStdLib_SwapEndianUint64

Prototype	
`uint64` **`VStdLib_SwapEndianUint64`** `( uint64 value )`	
Parameter	
`value [in]`	Unsigned 64-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified unsigned 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-100 VStdLib_SwapEndianUint64

4.2.10.6 VStdLib_SwapEndianSint64

Prototype	
`sint64` **`VStdLib_SwapEndianSint64`** `( sint64 value )`	
Parameter	
`value [in]`	Signed 64-bit value, whose endianness shall be swapped.
Return code	
	Value in the swapped endianness.
Functional Description	
Swaps the endianness of the specified signed 64-bit value.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined. This service is implemented externally if `VSTDLIB_USE_64BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON`.	
Call context	
ANY	

Table 4-101 VStdLib_SwapEndianSint64

4.2.11 Endianness conversion

4.2.11.1 VStdLib_ConvertEndianPlatformToBigUint16

Prototype	
`uint16 VStdLib_ConvertEndianPlatformToBigUint16 ( uint16 value )`	
Parameter	
`value [in]`	Unsigned 16-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified unsigned 16-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-102 VStdLib_ConvertEndianPlatformToBigUint16

4.2.11.2 VStdLib_ConvertEndianPlatformToBigSint16

Prototype	
`sint16 VStdLib_ConvertEndianPlatformToBigSint16 ( sint16 value )`	
Parameter	
`value [in]`	Signed 16-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified signed 16-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-103 VStdLib_ConvertEndianPlatformToBigSint16

4.2.11.3 VStdLib_ConvertEndianPlatformToLittleUint16

Prototype	
`uint16 VStdLib_ConvertEndianPlatformToLittleUint16 ( uint16 value )`	
Parameter	
`value [in]`	Unsigned 16-bit value according to the platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified unsigned 16-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-104 VStdLib_ConvertEndianPlatformToLittleUint16

4.2.11.4 VStdLib_ConvertEndianPlatformToLittleSint16

Prototype	
`sint16 VStdLib_ConvertEndianPlatformToLittleSint16 ( sint16 value )`	
Parameter	
`value [in]`	Signed 16-bit value according to platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified signed 16-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-105 VStdLib_ConvertEndianPlatformToLittleSint16

4.2.11.5 VStdLib_ConvertEndianBigToPlatformUint16

Prototype	
`uint16 VStdLib_ConvertEndianBigToPlatformUint16 ( uint16 value )`	
Parameter	
`value [in]`	Unsigned 16-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 16-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-106 VStdLib_ConvertEndianBigToPlatformUint16

4.2.11.6 VStdLib_ConvertEndianBigToPlatformSint16

Prototype	
`sint16 VStdLib_ConvertEndianBigToPlatformSint16 ( sint16 value )`	
Parameter	
`value [in]`	Signed 16-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified signed 16-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-107 VStdLib_ConvertEndianBigToPlatformSint16

4.2.11.7 VStdLib_ConvertEndianLittleToPlatformUint16

Prototype	
`uint16` **`VStdLib_ConvertEndianLittleToPlatformUint16`** `( uint16 value )`	
Parameter	
`value [in]`	Unsigned 16-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 16-bit value from little endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-108 VStdLib_ConvertEndianLittleToPlatformUint16

4.2.11.8 VStdLib_ConvertEndianLittleToPlatformSint16

Prototype	
`sint16` **`VStdLib_ConvertEndianLittleToPlatformSint16`** `( sint16 value )`	
Parameter	
`value [in]`	Signed 16-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified signed 16-bit value from little endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-109 VStdLib_ConvertEndianLittleToPlatformSint16

4.2.11.9 VStdLib_ConvertEndianPlatformToBigUint32

Prototype	
`uint32 VStdLib_ConvertEndianPlatformToBigUint32 ( uint32 value )`	
Parameter	
`value [in]`	Unsigned 32-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified unsigned 32-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-110 VStdLib_ConvertEndianPlatformToBigUint32

4.2.11.10 VStdLib_ConvertEndianPlatformToBigSint32

Prototype	
`sint32 VStdLib_ConvertEndianPlatformToBigSint32 ( sint32 value )`	
Parameter	
`value [in]`	Signed 32-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified signed 32-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-111 VStdLib_ConvertEndianPlatformToBigSint32

4.2.11.11 VStdLib_ConvertEndianPlatformToLittleUint32

Prototype	
`uint32 VStdLib_ConvertEndianPlatformToLittleUint32 ( uint32 value )`	
Parameter	
`value [in]`	Unsigned 32-bit value according to the platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified unsigned 32-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-112 VStdLib_ConvertEndianPlatformToLittleUint32

4.2.11.12 VStdLib_ConvertEndianPlatformToLittleSint32

Prototype	
`sint32 VStdLib_ConvertEndianPlatformToLittleSint32 ( sint32 value )`	
Parameter	
`value [in]`	Signed 32-bit value according to the platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified signed 32-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-113 VStdLib_ConvertEndianPlatformToLittleSint32

4.2.11.13 VStdLib_ConvertEndianBigToPlatformUint32

Prototype	
`uint32 VStdLib_ConvertEndianBigToPlatformUint32 ( uint32 value )`	
Parameter	
`value [in]`	Unsigned 32-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 32-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-114 VStdLib_ConvertEndianBigToPlatformUint32

4.2.11.14 VStdLib_ConvertEndianBigToPlatformSint32

Prototype	
`sint32 VStdLib_ConvertEndianBigToPlatformSint32 ( sint32 value )`	
Parameter	
`value [in]`	Signed 32-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified signed 32-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-115 VStdLib_ConvertEndianBigToPlatformSint32

4.2.11.15 VStdLib_ConvertEndianLittleToPlatformUint32

Prototype	
`uint32 VStdLib_ConvertEndianLittleToPlatformUint32 ( uint32 value )`	
Parameter	
`value [in]`	Unsigned 32-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 32-bit value from little endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-116 VStdLib_ConvertEndianLittleToPlatformUint32

4.2.11.16 VStdLib_ConvertEndianLittleToPlatformSint32

Prototype	
`sint32 VStdLib_ConvertEndianLittleToPlatformSint32 ( sint32 value )`	
Parameter	
`value [in]`	Signed 32-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness .
Functional Description	
Converts the specified signed 32-bit value from little endian to the platform endianness .	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant.	
Call context	
ANY	

Table 4-117 VStdLib_ConvertEndianLittleToPlatformSint32

4.2.11.17 VStdLib_ConvertEndianPlatformToBigUint64

Prototype	
`uint64 VStdLib_ConvertEndianPlatformToBigUint64 ( uint64 value )`	
Parameter	
`value [in]`	Unsigned 64-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified unsigned 64-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-118 VStdLib_ConvertEndianPlatformToBigUint64

4.2.11.18 VStdLib_ConvertEndianPlatformToBigSint64

Prototype	
`sint64 VStdLib_ConvertEndianPlatformToBigSint64 ( sint64 value )`	
Parameter	
`value [in]`	Signed 64-bit value according to the platform endianness, which shall be converted to big endian.
Return code	
	Value in big endian.
Functional Description	
Converts the specified signed 64-bit value from the platform endianness to big endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-119 VStdLib_ConvertEndianPlatformToBigSint64

4.2.11.19 VStdLib_ConvertEndianPlatformToLittleUint64

Prototype	
`uint64 VStdLib_ConvertEndianPlatformToLittleUint64 ( uint64 value )`	
Parameter	
`value [in]`	Unsigned 64-bit value according to the platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified unsigned 64-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-120 VStdLib_ConvertEndianPlatformToLittleUint64

4.2.11.20 VStdLib_ConvertEndianPlatformToLittleSint64

Prototype	
`sint64 VStdLib_ConvertEndianPlatformToLittleSint64 ( sint64 value )`	
Parameter	
`value [in]`	Signed 64-bit value according to the platform endianness, which shall be converted to little endian.
Return code	
	Value in little endian.
Functional Description	
Converts the specified signed 64-bit value from the platform endianness to little endian.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-121 VStdLib_ConvertEndianPlatformToLittleSint64

4.2.11.21 VStdLib_ConvertEndianBigToPlatformUint64

Prototype	
`uint64` **`VStdLib_ConvertEndianBigToPlatformUint64`** `( uint64 value )`	
Parameter	
`value [in]`	Unsigned 64-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 64-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-122 VStdLib_ConvertEndianBigToPlatformUint64

4.2.11.22 VStdLib_ConvertEndianBigToPlatformSint64

Prototype	
`sint64` **`VStdLib_ConvertEndianBigToPlatformSint64`** `( sint64 value )`	
Parameter	
`value [in]`	Signed 64-bit value according to big endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified signed 64-bit value from big endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-123 VStdLib_ConvertEndianBigToPlatformSint64

4.2.11.23 VStdLib_ConvertEndianLittleToPlatformUint64

Prototype	
`uint64 VStdLib_ConvertEndianLittleToPlatformUint64 ( uint64 value )`	
Parameter	
`value [in]`	Unsigned 64-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified unsigned 64-bit value from little endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-124 VStdLib_ConvertEndianLittleToPlatformUint64

4.2.11.24 VStdLib_ConvertEndianLittleToPlatformSint64

Prototype	
`sint64 VStdLib_ConvertEndianLittleToPlatformSint64 ( sint64 value )`	
Parameter	
`value [in]`	Signed 64-bit value according to little endian, which shall be converted to the platform endianness.
Return code	
	Value in the platform endianness.
Functional Description	
Converts the specified signed 64-bit value from little endian to the platform endianness.	
Particularities and Limitations	
> This function is synchronous. > This function is reentrant. > Configuration Variant(s): This service is only available if `PLATFORM_SUPPORT_SINT64_UINT64` is defined.	
Call context	
ANY	

Table 4-125 VStdLib_ConvertEndianLittleToPlatformSint64

4.3 Services used by VStdLib

In the following table services provided by other components, which are used by the VStdLib are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	`Det_ReportError()`
(Compiler) Library	Refer to section 5.2.2

Table 4-126 Services used by VStdLib

5 Configuration

In the VStdLib the attributes can be configured with the following method:

> Configuration manually in header files, for a detailed description see 5.2

5.1 Configuration Variants

The VStdLib supports following configuration variants:

> VARIANT-PRE-COMPILE

5.2 Configuration manually in Header Files

The configuration of the VStdLib is done statically within the file “VStdLib_Cfg.h”.

5.2.1 General configuration

Attribute Name	Values Default value is typed bold	Description
VSTDLIB_USE_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, the memory initialization, memory copy and memory comparison APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.1. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_MEM_CPY_ENDIAN_CONVERSION_LIBRARY_FUNCTION	STD_ON **STD_OFF**	If set to STD_ON, the memory copy with endianness swap API is mapped to an external function (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.1. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_MEM_CHECK_LIBRARY_FUNCTION	STD_ON **STD_OFF**	If set to STD_ON, the memory check for pattern API is mapped to an external function (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.1. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all bit copy sawtooth APIs are mapped to external functions (e.g. compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.1. If set to STD_OFF, the generic functions provided by the VStdLib are used.

Attribute Name	Values Default value is typed bold	Description
VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all bit copy monotone APIs are mapped to external functions (e.g. compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.1. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_RUNTIME_OPTIMIZATION	**STD_ON** STD_OFF	If set to STD_ON, optimized routines are used to increase the performance of memory initialization, memory copy and memory comparison services (increases code size). If set to STD_OFF, code efficient routines are used that increase runtime. This setting is only relevant if VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_OFF.
VSTDLIB_USE_JUMPTABLES	**STD_ON** STD_OFF	If set to STD_ON, jump tables are used to increase the performance of the memory initialization and memory copy services (runtime efficient in general). If set to STD_OFF, the jump tables are replaced by loops. Use this setting if the compiler generates no efficient code for the jump tables. This setting is only relevant if VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_OFF and VSTDLIB_RUNTIME_OPTIMIZATION == STD_ON.
VSTDLIB_DEV_ERROR_DETECT	STD_ON **STD_OFF**	If set to STD_ON, the development error detection is enabled. In this case the pointer arguments of all global module functions provided by the VStdLib are checked. If any NULL_PTR is passed, these functions return without performing any action.
VSTDLIB_DEV_ERROR_REPORT	STD_ON **STD_OFF**	If set to STD_ON, the development error reporting is enabled (requires VSTDLIB_DEV_ERROR_DETECT == STD_ON). In this case the function Det_ReportError() is called if any error is detected.
VSTDLIB_VERSION_INFO_API	STD_ON **STD_OFF**	If set to STD_ON, the function VStdLib_GetVersionInfo() is provided.
VSTDLIB_DUMMY_STATEMENT(v)	e.g. (v) = (v) **(void)(v)**	Expression that is used for dummy statements to avoid compiler warnings about unused identifiers. Leave this definition empty to disable the usage of dummy statements.

Attribute Name	Values Default value is typed bold	Description
VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 8-bit bit search APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.2. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 16-bit bit search APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.2. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 32-bit bit search APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.2. If set to STD_OFF the generic functions provided by the VStdLib are used.
VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 64-bit bit search APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.2. If set to STD_OFF, the generic functions provided by the VStdLib are used. The setting of this define is only allowed if PLATFORM_SUPPORT_SINT64_UINT64 is defined.
VSTDLIB_USE_16BIT_DE_SERIALIATION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 16-bit deserialization and serialization APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.3. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_32BIT_DE_SERIALIATION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 32-bit deserialization and serialization APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.3. If set to STD_OFF, the generic functions provided by the VStdLib are used.

Attribute Name	Values Default value is typed bold	Description
VSTDLIB_USE_64BIT_DE_SERIALIATION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 64-bit deserialization and serialization APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.3. If set to STD_OFF, the generic functions provided by the VStdLib are used. The setting of this define is only allowed if PLATFORM_SUPPORT_SINT64_UINT64 is defined.
VSTDLIB_USE_16BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 16-bit endianness swap APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.4. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_32BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 32-bit endianness swap APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.4. If set to STD_OFF, the generic functions provided by the VStdLib are used.
VSTDLIB_USE_64BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS	STD_ON **STD_OFF**	If set to STD_ON, all 64-bit endianness swap APIs are mapped to external functions (e.g., compiler libraries or other implementations that are optimized for the target platform). This requires additional configuration, refer to section 5.2.2.4. If set to STD_OFF, the generic functions provided by the VStdLib are used. The setting of this define is only allowed if PLATFORM_SUPPORT_SINT64_UINT64 is defined.

Table 5-1 General configuration

5.2.2 Additional configuration when using library functions

5.2.2.1 Memory services

If `VSTDLIB_USE_LIBRARY_FUNCTIONS == STD_ON`, it is necessary to specify library functions to be used for the memory initialization, memory copy and memory comparison APIs.

If `VSTDLIB_USE_MEM_CPY_ENDIAN_CONVERSION_LIBRARY_FUNCTION == STD_ON`, it is necessary to specify a library function to be used for the memory copy with endianness swap API.

If `VSTDLIB_USE_MEM_CHECK_LIBRARY_FUNCTION == STD_ON`, it is necessary to specify a library function to be used for the memory check for pattern API.

If `VSTDLIB_USE_BITCPYSAWTOOTH_LIBRARY_FUNCTIONS == STD_ON`, it is necessary to specify library functions to be used for the bit copy sawtooth APIs.

If `VSTDLIB_USE_BITCPYMONOTONE_LIBRARY_FUNCTIONS == STD_ON`, it is necessary to specify library functions to be used for the bit copy monotone APIs.

See the corresponding section in “VStdLib_Cfg.h” for details and an example mapping.



Caution

If the external functionality is not able to handle more than 65535 bytes it is necessary to define `VSTDLIB_SUPPORT_LARGE_DATA` to `STD_OFF`.

The specified functions must behave synchronously.

It has to be ensured that the specified memory initialization and memory copy functions are able to copy from and to all memory locations independently of the pointer length.

5.2.2.2 Bit search services

If VSTDLIB_USE_8BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON, VSTDLIB_USE_16BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON, VSTDLIB_USE_32BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON or VSTDLIB_USE_64BIT_SEARCH_LIBRARY_FUNCTIONS == STD_ON, it is necessary to specify library functions to be used for the bit width dependent bit search APIs. See the corresponding section in “VStdLib_Cfg.h” for details.



Caution
The 64-bit bit search APIs can only be overwritten if the target platform supports the data type UINT64 (PLATFORM_SUPPORT_SINT64_UINT64 is defined in Platform_Types.h).

5.2.2.3 Deserialization and serialization services

If VSTDLIB_USE_16BIT_DE_SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON, VSTDLIB_USE_32BIT_DE_ SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON or VSTDLIB_USE_64BIT_DE_ SERIALIZATION_LIBRARY_FUNCTIONS == STD_ON, it is necessary to specify library functions to be used for the bit width dependent deserialization and serialization APIs. See the corresponding section in “VStdLib_Cfg.h” for details.



Caution
The 64-bit deserialization and serialization APIs can only be overwritten if the target platform supports the data types SINT64 and UINT64 (PLATFORM_SUPPORT_SINT64_UINT64 is defined in Platform_Types.h).

5.2.2.4 Endianness conversion services

If VSTDLIB_USE_16BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON, VSTDLIB_USE_32BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON or VSTDLIB_USE_64BIT_ENDIAN_CONVERSION_LIBRARY_FUNCTIONS == STD_ON, it is necessary to specify library functions to be used for the bit width dependent endianness swap APIs. See the corresponding section in “VStdLib_Cfg.h” for details.



Caution
The 64-bit endianness swap APIs can only be overwritten if the target platform supports the data types SINT64 and UINT64 (PLATFORM_SUPPORT_SINT64_UINT64 is defined in Platform_Types.h).

6 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Default Error Tracer
ECU	Electronic Control Unit
MCU	Microcontroller Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
VStdLib	Vector Standard Library

Table 6-1 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com